AI / ML FOUNDATIONS • STAGE 1 • COMPUTATIONAL LAYER

NumPy
Master Notes

PART 3

Broadcasting + Comparisons, Masks and Boolean Logic

Based on: Python Data Science Handbook, Chapter 2 (Blocks 5 and 6)

Theory | Examples | Visual Outputs | Practice Problems | Complete Answer Key

WHAT IS INSIDE

- 1. The big idea of broadcasting
- 2. Both arrays stretch: np.newaxis
- 3. The 3 rules of broadcasting
- 4. The rules on real code
- 5. Fixing a failed broadcast
- 6. Broadcasting in real use
- 7. Comparisons give Boolean arrays
- 8. Counting and checking
- 9. Combining conditions: & | ~
- 10. Boolean masks: selecting values
- 11. and / or versus & / |
- 12. Revision zone and final paper

Slow-paced notes: read, understand, then practise. Answers are at the very end.

Prepared for Abhijeet • October 2026

How to use these notes

These notes cover the two blocks you studied from the Python Data Science Handbook: **Block 5: Computation on Arrays: Broadcasting** and **Block 6: Comparisons, Masks, and Boolean Logic**. Every topic follows the same order as in Part 2 of your notes: the idea first, then a real-life picture, then code, then the exact output you will see in Colab. After every part you get practice problems. All correct answers are at the very end, in the Answer Key, with the real output of every program.

Because you said you cannot memorise the syntax of every function, each function carries a priority tag so you know where to spend your energy. Things the book mentions that are not worth your time right now were left out on purpose (the list is in the Revision Zone). From the book's Block 5 and 6 material, the **three broadcasting rules**, `np.newaxis`, **masks** and the `&` **versus** `and` rule are the parts that build real intuition.

Tag	Meaning	What to do
MUST KNOW	You will type this almost every day in AI/ML work.	Practise until your fingers know it.
GOOD TO KNOW	Used regularly, but not every day.	Know what it does and roughly how it looks.
LOOK UP	Rare, or easy to search when needed.	Remember that it exists. Do not memorise.

The coloured boxes you will meet:

ANALOGY	A real-life picture so the idea sticks.
MEMORY TRICK	A short trick to remember something forever.
INTERVIEW FLAG	The kind of question interviewers and viva examiners love to ask.
INTERVIEW POINT	Write or say this in an answer for full marks.
IMPORTANT	Pay attention: easy to get wrong.
COMMON TRAP	A mistake almost every beginner makes. NumPy never warns you about most of these.
TRY IT	Type this into Colab and watch it happen yourself.
WHY IT MATTERS	Where this idea shows up in real AI/ML work.
VERSION NOTE	Newer NumPy differs from the book in a small way.
NOTE	Extra detail that helps understanding.

The practice problem tags:

Tag	What you must do
PREDICT THE OUTPUT	Read the code and say exactly what Python will print. Do it in your head first, then run it.
FIND THE BUGS	The code looks fine but is broken or gives a wrong result. Find every mistake.
BUILD IT	Write the code yourself from a description.
THINK IT THROUGH	Reason about why. No memorising possible.
CALCULATE	Work out sizes and numbers step by step.
READ & DECODE	Read a line of NumPy and explain it in plain words.
THEORY	Interview-style written answers.

STUDY RULE

Do not read these notes like a story book only. Open Colab, type the examples, change one value at a time and watch what changes. NumPy is learnt with the fingers, not only the eyes. Before you look at any answer, **say out loud what got stretched** (broadcasting) or **what the mask keeps** (masks). All outputs in these notes were produced by running the code (NumPy 2.4.4), so what you see on the page is what you should see on your screen. The rainfall data is simulated (the book's CSV file is not available here), so those numbers differ from the book while the techniques are identical.

Your Block 5 and 6 topic map

#	Topic	Why it matters in AI/ML	Priority
1	The big idea of broadcasting	Why shapes need not match; bias in neural networks	**MUST KNOW**
2	Both arrays stretch: `np.newaxis`	Pairwise tables and grids without loops	**MUST KNOW**
3	The three rules	Predict any shape; read any shape error	**MUST KNOW**
4	The rules on real code	Debugging "could not be broadcast"	**MUST KNOW**
5	Fixing a failed broadcast	`keepdims`, `reshape(-1, 1)`, silent bugs	**MUST KNOW**
6	Broadcasting in real use	Centring, standardisation, 2D grids	**MUST KNOW**
7	Comparisons give Boolean arrays	Accuracy, thresholds, ReLU	**MUST KNOW**
8	Counting and checking	`np.sum`, `np.mean`, `np.any`, `np.all`	**MUST KNOW**
9	Combining conditions	Filtering with several rules	**MUST KNOW**
10	Boolean masks	Selecting classes, outliers, mistakes	**MUST KNOW**
11	`and`/`or` versus `&`/`|`	The most common array error message	**MUST KNOW**
12	Revision zone	Cheat sheet, interview, error radar	**GOOD TO KNOW**

Contents

Broadcasting: The Big Idea . . . 7

1.1 The problem with same-shape-only . . . 7
1.2 What broadcasting is . . . 7
1.3 Scalar and array . . . 8
1.4 The stretching is only imagined . . . 8
1.5 1D array with a 2D array . . . 9
1.6 Why this matters in AI/ML . . . 9

Both Arrays Stretch . . . 12

2.1 When both arrays stretch . . . 12
2.2 What is np.newaxis? . . . 13
2.3 Three ways to say the same thing . . . 14
2.4 Why this matters: all pairs without a loop . . . 14

The 3 Rules of Broadcasting . . . 16

3.1 NumPy compares shapes from the right . . . 16
3.2 The compact version . . . 17
3.3 The rules in pictures . . . 17
3.4 A quick table to practise on . . . 17
3.5 Checking without running the operation . . . 18

The Rules on Real Code . . . 19

4.1 Example 1: 2D plus 1D (the bias pattern) . . . 19
4.2 Example 2: both arrays stretch . . . 19
4.3 Example 3: the failure . . . 20
4.4 How to read the error and fix it . . . 21
4.5 It works for every binary ufunc . . . 21

Fixing a Failed Broadcast . . . 23

5.1 The problem . . . 23
5.2 The fix: turn w into a column . . . 23
5.3 Two ways to make a column . . . 24
5.4 The decision rule . . . 24
5.5 The silent trap on square arrays . . . 24

Broadcasting in Real Use . . . 26

6.1 Centring each column . . . 26
6.2 Centring each row: the silent bug and keepdims . . . 27
6.3 Scaling each column by its maximum . . . 28
6.4 A function over a whole grid . . . 28
6.5 The standardisation line, decoded . . . 29

Comparisons Give Boolean Arrays . . . 32

7.1 The problem masks solve . . . 32
7.2 The six comparison operators . . . 32
7.3 Compound expressions . . . 33
7.4 Any shape, and broadcasting works too . . . 33
7.5 The ufuncs hiding behind the operators . . . 34
7.6 The payoff: accuracy in one line . . . 35
7.7 Never compare floats with == . . . 35

Counting and Checking . . . 37

8.1 Counting the True values . . . 37
8.2 Counting per row and per column . . . 38
8.3 The fraction of True values . . . 38
8.4 np.any and np.all . . . 39
8.5 A useful ML check: is there missing data? . . . 39
8.6 The built-in sum, any and all trap . . . 40

Combining Conditions . . . 42

9.1 Why one condition is not enough . . . 42
9.2 AND, OR, NOT in action . . . 42
9.3 The brackets are mandatory . . . 43
9.4 De Morgan's law (two ways to say the same thing) . . . 44
9.5 The ~ trap on integer arrays . . . 44
9.6 The ufuncs behind the operators . . . 44

Boolean Masks: Selecting Values . . . 47

10.1 From counting to selecting . . . 47
10.2 Selecting from a 1D array . . . 48
10.3 Statistics on the selected values . . . 48
10.4 Named masks and the rainfall example . . . 49
10.5 Selecting whole rows with a 1D mask . . . 50
10.6 Changing values through a mask . . . 50
10.7 x[mask] makes a copy . . . 51

10.8 A mask of the wrong length . . . 51

and / or versus & / | . . . 54

11.1 Two kinds of "and" . . . 54

11.2 On plain numbers . . . 54

11.3 On NumPy Boolean arrays . . . 55

11.4 The fix, and the legitimate single answer . . . 55

Revision Zone . . . 58

12.1 Function cheat sheet . . . 58

12.2 Topic checklist . . . 59

12.3 Interview quick-fire: 20 questions with model answers . . . 60

12.4 The error radar: check these before you trust your output . . . 61

12.5 Final mixed paper . . . 62

Answer Key . . . 64

Answers: Part 1: Broadcasting: The Big Idea . . . 64

Answers: Part 2: Both Arrays Stretch . . . 65

Answers: Part 3: The 3 Rules of Broadcasting . . . 67

Answers: Part 4: The Rules on Real Code . . . 68

Answers: Part 5: Fixing a Failed Broadcast . . . 69

Answers: Part 6: Broadcasting in Real Use . . . 70

Answers: Part 7: Comparisons Give Boolean Arrays . . . 72

Answers: Part 8: Counting and Checking . . . 73

Answers: Part 9: Combining Conditions . . . 75

Answers: Part 10: Boolean Masks: Selecting Values . . . 77

Answers: Part 11: and / or versus & / | . . . 79

Answers: Final Mixed Paper (Blocks 5 and 6) . . . 80

PART 1

1 Broadcasting: The Big Idea

Block 5, first section: operating on arrays of different sizes

IN THIS PART YOU WILL LEARN

- ✓ Why "same shape only" is too limiting, and what broadcasting fixes
- ✓ Broadcasting a single number (a scalar) against a whole array
- ✓ Broadcasting a 1D array against a 2D array (the neural-network bias pattern)
- ✓ Why the stretching is only imagined, never really copied
- ✓ The habit that makes shape errors easy: always say **what stretched**

1.1 The problem with same-shape-only

In Block 3 you saw that arithmetic on two arrays works **position by position**. That only works when both arrays have the **same shape**, so every element has a partner:

PYTHON • type this in Colab

```
a = np.array([0, 1, 2])
b = np.array([5, 5, 5])
print(a + b)
```

▶ OUTPUT

```
[5 6 7]
```

But suppose you only want to **add 5 to every element**. Writing `[5, 5, 5]` by hand every time is annoying, and for an array with a million elements it is impossible. We need a way to say "just add 5" and let NumPy handle the rest.

1.2 What broadcasting is

Broadcasting is a set of rules that lets NumPy apply an operation (like `+`, `-`, `*`, `/`) to arrays of **different shapes**, by automatically **stretching the smaller array** so it matches the bigger one.

PYTHON • type this in Colab

```
a = np.array([0, 1, 2])
print(a + 5)
```

▶ OUTPUT

```
[5 6 7]
```

ANALOGY

Bonus marks for the whole class. The teacher announces "everyone gets +5 marks". She does not write thirty separate slips saying "+5"; she says it once and it applies to every student. Broadcasting is that single announcement: one number, applied to every element.

INTERVIEW FLAG

"What is broadcasting in NumPy?"
Model answer: broadcasting is the set of rules NumPy uses to perform element-wise operations on arrays of different shapes, by virtually stretching the smaller array to match the larger one, without actually copying the data.



dashed cells are never stored in memory: they only help you think

A scalar is stretched (in imagination) into a full array, then added element by element.

1.3 Scalar and array

A scalar is a **zero-dimensional** array: just one number. Combined with an array, it is applied to **every** element. You already used this idea all through Block 3 (`x * 2`, `x + 5`, `1.0 / values`). Now it has a name.

PYTHON • type this in Colab

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])
print(M * 10)
```

▶ OUTPUT

```
[[10 20 30]
 [40 50 60]]
```

What got stretched? The scalar `10` was stretched into a 2×3 array of tens, `[[10, 10, 10], [10, 10, 10]]`, and then multiplied element by element.

MEMORY TRICK

Always finish this sentence: "___ got stretched, across/down ___." From this Part onward, saying out loud what stretched is how you will predict results and debug shape errors.

1.4 The stretching is only imagined

Think of broadcasting as stretching, but know that NumPy does **not** really build the bigger array. The extra copies are never stored in memory; NumPy just reuses the same numbers. That is why broadcasting is fast and uses almost no extra memory, even on huge arrays.

IMPORTANT

Stretching is a mental model, not what happens in memory. The dashed cells in the diagrams are imaginary. If you add 5 to an array of one million numbers, NumPy does **not** create one million extra 5s.

If you want to **see** the imagined stretched array, `np.broadcast_to` shows it **LOOK UP**:

PYTHON • type this in Colab

```
a = np.array([0, 1, 2])
print(np.broadcast_to(a, (3, 3)))
```

▶ OUTPUT

```
[[0 1 2]
 [0 1 2]
 [0 1 2]]
```

1.5 1D array with a 2D array

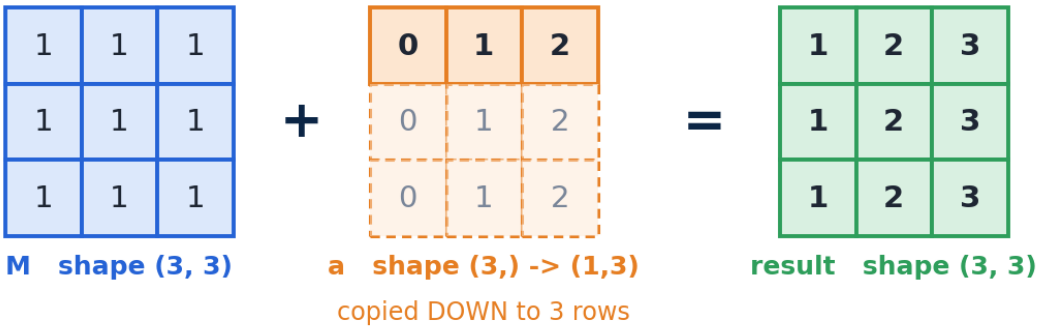
The same idea extends to higher dimensions. Here a 1D array is added to every row of a 2D array:

PYTHON • type this in Colab

```
M = np.ones((3, 3))
a = np.array([0, 1, 2])
print(M + a)
```

▶ OUTPUT

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```



The row [0 1 2] is copied (in imagination) down to every row of M, then added.

What got stretched? The 1D array `a` was stretched **down** to 3 rows.

1.6 Why this matters in AI/ML

WHY IT MATTERS

A neural-network layer computes `X @ W + b`. The part `X @ W` is a matrix (one row per sample), and the **bias** `b` is a single row. Adding `b` to every row of the matrix is broadcasting. The same trick is used when normalising data, computing distances and in attention in transformers. PyTorch uses exactly the same rules.

Here is the bias pattern with small numbers (pretend `Z` is the output of `X @ W` for 2 samples and 3 neurons):

PYTHON • type this in Colab

```
Z = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])
print(Z + b)
```

▶ OUTPUT

```
[[11 22 33]
 [14 25 36]]
```

Every sample (row) gets the same bias vector added. One line, no loop.

✎ PRACTICE PROBLEMS | Part 1: Broadcasting: The Big Idea

Q1 **PREDICT THE OUTPUT**

Predict the output without running it. Then say **what got stretched**.

PYTHON • type this in Colab

```
import numpy as np
M = np.array([[1, 2, 3],
              [4, 5, 6]])
print(M * 10)
```

Q2 **PREDICT THE OUTPUT**

Predict both lines.

PYTHON • type this in Colab

```
import numpy as np
print(np.array([1, 2, 3]) + 100)
print(np.array([[1, 2], [3, 4]]) - 1)
```

Q3 **THINK IT THROUGH**

You add `5` to an array of one million numbers. How many extra copies of `5` does NumPy store in memory? Why do we still talk about "stretching"?

Q4 **PREDICT THE OUTPUT**

Predict the output. What got stretched, and in which direction?

PYTHON • type this in Colab

```
import numpy as np
M = np.ones((2, 3))
a = np.array([10, 20, 30])
print(M + a)
```

Q5 **BUILD IT**

`Z` has shape (4, 3) and `b = np.array([1, 2, 3])`. Write one line that adds `b` to **every row** of `Z`, and give the output.

PYTHON • type this in Colab

```
import numpy as np
Z = np.zeros((4, 3))
b = np.array([1, 2, 3])
```

Q6 **FIND THE BUGS**

This code works but is slow and long. Replace it with a single vectorized line.

PYTHON • type this in Colab

```
import numpy as np
Z = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([10, 20, 30])
out = np.empty(Z.shape)
for i in range(Z.shape[0]):
    for j in range(Z.shape[1]):
        out[i, j] = Z[i, j] + b[j]
```

Q7 **THEORY**

Define broadcasting in two lines and give one use of it in AI/ML. **[3 marks]**

2

PART 2

Both Arrays Stretch

Block 5: row + column makes a grid, and np.newaxis

IN THIS PART YOU WILL LEARN

- ✓ What happens when **both** arrays need stretching
- ✓ How a column (n,1) and a row (1,m) combine into a grid (n,m)
- ✓ What `np.newaxis` does, and its aliases `None` and `reshape`
- ✓ How to read shapes: `(3,)` versus `(3,1)` versus `(1,3)`
- ✓ Pairwise combinations without loops

2.1 When both arrays stretch

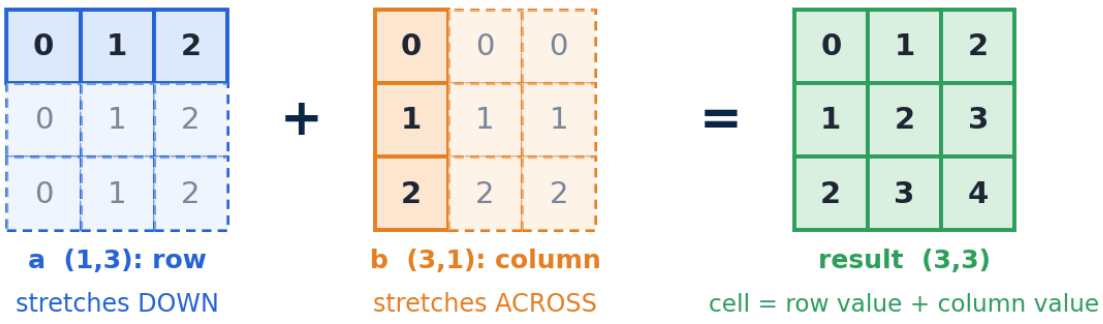
Last Part only one array stretched. Sometimes **both** do:

PYTHON • type this in Colab

```
a = np.arange(3)
b = np.arange(3)[:, np.newaxis]
print(a)
print(b)
print(a + b)
```

▶ OUTPUT

```
[0 1 2]
[[0]
 [1]
 [2]]
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```



The row stretches down, the column stretches across, and every cell becomes row value + column value.

Here `a` has shape `(3,)` (treated as a row, `(1, 3)`) and `b` has shape `(3, 1)` (a column). The row is stretched **down** to 3 rows, the column is stretched **across** to 3 columns. Both become `(3, 3)`, and the result is a **grid**: cell `[i, j]` equals `b[i] + a[j]`.

MEMORY TRICK **A column `(n, 1)` combined with a row `(1, m)` always gives a grid `(n, m)`.**

2.2 What is np.newaxis?

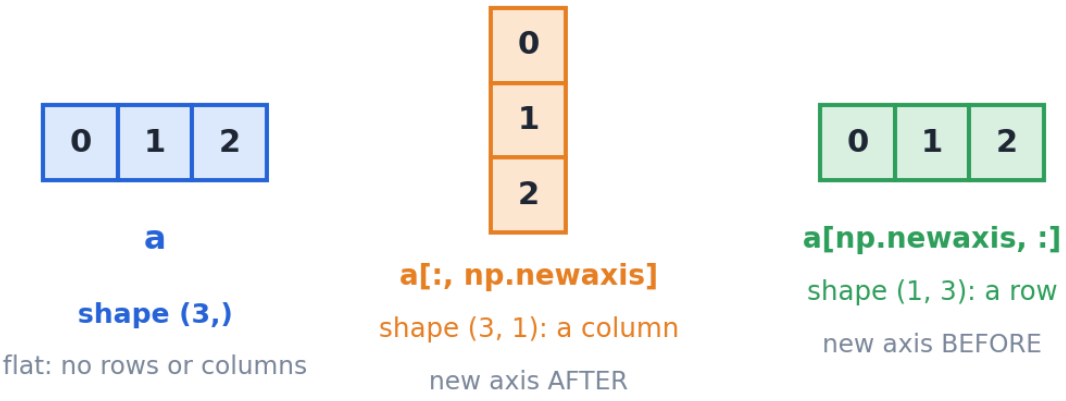
`np.newaxis` inserts a **new dimension of size 1** at the position where you write it. This is the most useful broadcasting tool you will use.

PYTHON • type this in Colab

```
a = np.arange(3)
print(a.shape)
print(a[:, np.newaxis].shape)
print(a[np.newaxis, :].shape)
```

▶ OUTPUT

```
(3,)
(3, 1)
(1, 3)
```



The same three numbers in three different shapes.

The `:` means "keep this axis exactly as it is". `np.newaxis` goes in the position where you want the **new** axis.

You write	Shape you get	It looks like
`a`	`(3,)`	flat: just 3 numbers
`a[:, np.newaxis]`	`(3, 1)`	a **column**: 3 rows, 1 column
`a[np.newaxis, :]`	`(1, 3)`	a **row**: 1 row, 3 columns

MEMORY TRICK

`a[:, np.newaxis]` puts the new axis **after** the colon, so the array **stands up** as a column.
`a[np.newaxis, :]` puts it **before**, so it **lies down** as a row.

2.3 Three ways to say the same thing

PYTHON • type this in Colab

```
a = np.arange(3)
print(a[:, np.newaxis].shape)
print(a[:, None].shape)
print(a.reshape(3, 1).shape)
print(a.reshape(-1, 1).shape)
```

▶ OUTPUT

```
(3, 1)
(3, 1)
(3, 1)
(3, 1)
```

Style	Notes	Priority
`a[:, np.newaxis]`	the readable, explicit way	**MUST KNOW**
`a[:, None]`	identical: `np.newaxis` is just another name for `None`. Very common in real code	**MUST KNOW**
`a.reshape(-1, 1)`	the `-1` means "work out this size yourself". Everywhere in scikit-learn	**MUST KNOW**

2.4 Why this matters: all pairs without a loop

WHY IT MATTERS

Any time you need **every combination** of two lists (all pairwise distances, an attention score for every query against every key, a multiplication table), you write one column against one row. No loops.

PYTHON • type this in Colab

```
points = np.array([1, 4, 6])
diff = points[:, None] - points[None, :]
print(diff)
print(np.abs(diff))
```

▶ OUTPUT

```
[[ 0 -3 -5]
 [ 3 0 -2]
 [ 5 2 0]]
[[0 3 5]
 [3 0 2]
 [5 2 0]]
```

Row `i`, column `j` holds `points[i] - points[j]`. Taking `np.abs` of it gives the distance between every pair of points: a 3×3 table from a single line.

✎ PRACTICE PROBLEMS | Part 2: Both Arrays Stretch

Q1 **PREDICT THE OUTPUT**

Give the shapes of `a` and `b`, the shape of the result, and the full output.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20])[:, np.newaxis]
print(a.shape, b.shape)
print(a + b)
```

Q2 **CALCULATE**

For `a = np.arange(4)`, give the shape of `a[:, np.newaxis]` and of `a[np.newaxis, :]`.

PYTHON • type this in Colab

```
import numpy as np
a = np.arange(4)
```

Q3 **BUILD IT**

Build a 5×5 multiplication table (rows and columns both 1 to 5) in one line, using a column and a row.

Q4 **THINK IT THROUGH**

Why does adding a `(3,)` array and a `(3, 1)` array give a 3×3 **grid** instead of 3 sums?

Q5 **READ & DECODE**

`x` has shape `(5,)` and `y` has shape `(7,)`. What does `x[:, None] - y[None, :]` compute, and what is the shape of the result?

Q6 **FIND THE BUGS**

This crashes. Explain why and fix it so you get a 3×4 grid of sums.

PYTHON • type this in Colab

```
import numpy as np
a = np.arange(3)
b = np.arange(4)
print(a + b)
```

Q7 **THEORY**

Explain `np.newaxis` with a before/after shape example and say why it is useful for broadcasting. **[3 marks]**

PART 3

3 The 3 Rules of Broadcasting

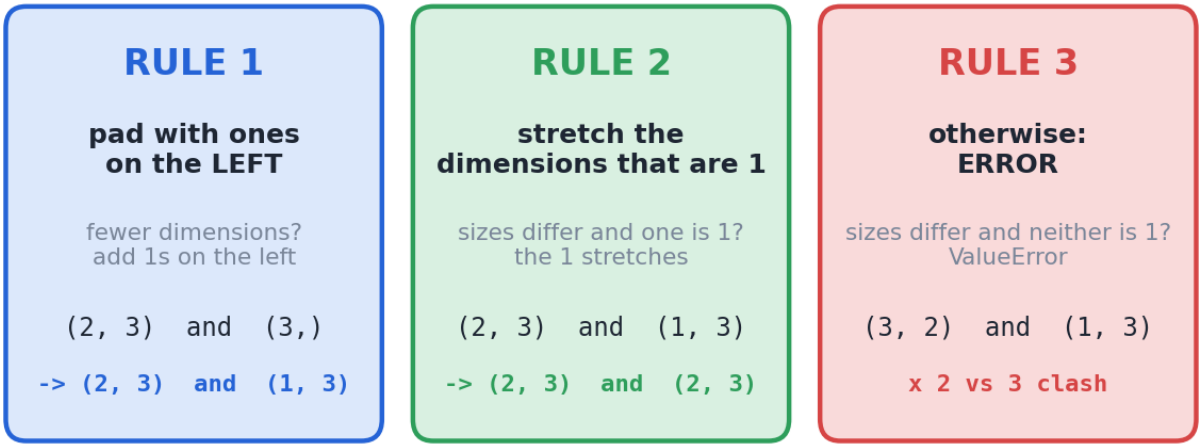
Block 5: pad left, stretch ones, otherwise error

IN THIS PART YOU WILL LEARN

- ✓ The exact three rules NumPy follows, in order
- ✓ The compact check: from the right, sizes must be **equal or one of them is 1**
- ✓ Why padding goes on the **left only**
- ✓ How to predict the result shape (or the error) on paper
- ✓ Checking compatibility with `np.broadcast_shapes`

3.1 NumPy compares shapes from the right

So far you have seen broadcasting work. Now you need the exact procedure NumPy follows, because this is what lets you **predict** shapes and **debug** errors. NumPy compares the two shapes **from right to left**, using three rules in order.



The three rules, each with a tiny example.

Rule	What it says	Example
Rule 1	If the arrays have different numbers of dimensions, pad the shape of the one with **fewer** dimensions with ones on its **left**.	`(2,3)` and `(3,)` becomes `(2,3)` and `(1,3)`
Rule 2	If sizes differ in a dimension and one of them is **1**, that one is stretched to match the other.	`(2,3)` and `(1,3)` becomes `(2,3)` and `(2,3)`
Rule 3	If sizes differ and **neither is 1**, raise a `ValueError`.	`(3,2)` and `(1,3)`: 2 vs 3, error

ANALOGY

Adding two numbers on paper. You line them up from the **right** (units under units, tens under tens). If one number is shorter, you imagine leading zeros on its **left**. Broadcasting lines shapes up the same way: from the right, with imaginary 1s on the left.

3.2 The compact version

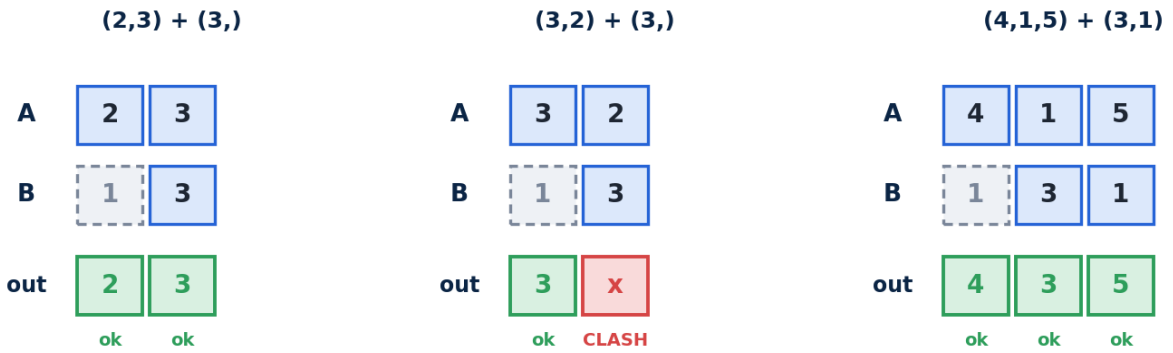
You can skip the three-step language and use one check. Going **from the right**, for each dimension the two sizes must be either:

- **equal**, or
- **one of them is 1**

If every dimension passes, the result shape takes the **larger** size in each position. If any dimension fails, you get an error.

MEMORY TRICK **Pad left, stretch ones, else error.**

3.3 The rules in pictures



Dashed grey boxes are the 1s added by Rule 1. Green means that dimension passes, red means it clashes.

3.4 A quick table to practise on

Shape A	Shape B	Compatible?	Result
`(3,)`	`(3,)`	Yes	`(3,)`
`(2, 3)`	`(3,)`	Yes	`(2, 3)`
`(3, 1)`	`(1, 3)`	Yes	`(3, 3)`
`(2, 3)`	`(2,)`	**No**	error: 3 vs 2
`(4, 1, 5)`	`(3, 1)`	Yes	`(4, 3, 5)`

Check the last row yourself: pad `(3, 1)` to `(1, 3, 1)`. Then compare from the right: 5 vs 1, 1 vs 3, 4 vs 1. All pass, so the result is `(4, 3, 5)`.

COMMON TRAP **Padding goes on the LEFT only, never the right.** People expect `(2, 3)` + `(2,)` to work because "the 2s match". But `(2,)` becomes `(1, 2)`, so the last dimension is 3 vs 2, which is an error. If you want the padding on the right, you do it yourself with `np.newaxis` (next Part).

3.5 Checking without running the operation

`np.broadcast_shapes` takes shapes (not arrays) and returns the result shape, or raises the same error. It is handy for checking your paper answers **GOOD TO KNOW**:

PYTHON • type this in Colab

```
print(np.broadcast_shapes((2, 3), (3,)))
print(np.broadcast_shapes((4, 1), (1, 6)))
print(np.broadcast_shapes((3, 2), (3,)))
```

▶ OUTPUT

```
(2, 3)
(4, 6)
ValueError: shape mismatch: objects cannot be broadcast to a single shape. Mismatch is between
arg 0 with shape (3, 2) and arg 1 with shape (3,).
```

✎ PRACTICE PROBLEMS | Part 3: The 3 Rules of Broadcasting

Q1 **CALCULATE**

For each pair, write the padded shape, say whether it is compatible, and give the result shape if it is. (a) `(5, 3)` and `(3,)` (b) `(5, 3)` and `(5,)` (c) `(4, 1)` and `(1, 6)` (d) `(2, 3, 4)` and `(3, 1)`

Q2 **CALCULATE**

Same task for: (a) `(4, 1)` and `(4,)` (b) `(1,)` and `(5, 3)` (c) `(8, 1, 6, 1)` and `(7, 1, 5)` (d) `(3, 4)` and `(4, 3)`.

Q3 **THINK IT THROUGH**

Why does `(3, 2) + (3,)` fail even though both arrays contain a 3?

Q4 **PREDICT THE OUTPUT**

Without running it: what is the shape of `np.ones((4, 1)) + np.arange(4)`? Then check.

PYTHON • type this in Colab

```
import numpy as np
print((np.ones((4, 1)) + np.arange(4)).shape)
```

Q5 **READ & DECODE**

State the compact check that decides compatibility, in one sentence, and say how the result shape is chosen.

Q6 **THEORY**

State the three rules of broadcasting in your own words, with one example for each. **[5 marks]**

PART 4

4 The Rules on Real Code

Block 5: the book's three examples, and how to read the error

IN THIS PART YOU WILL LEARN

- ✓ Apply the three rules step by step to real arrays
- ✓ See a broadcast that works, one that needs both arrays to stretch, and one that fails
- ✓ Read the error message `operands could not be broadcast together`
- ✓ A 3-step routine to debug any shape clash
- ✓ Broadcasting works for **every** binary ufunc, not just `+`

4.1 Example 1: 2D plus 1D (the bias pattern)

PYTHON • type this in Colab

```
M = np.ones((2, 3))
a = np.arange(3)
print(M.shape, a.shape)
print(M + a)
```

▶ OUTPUT

```
(2, 3) (3,)
[[1. 2. 3.]
 [1. 2. 3.]]
```

Step	Shape of M	Shape of a	What happened
Start	`(2, 3)`	`(3,)`	
Rule 1	`(2, 3)`	`(1, 3)`	`a` has fewer dimensions: pad a 1 on the left
Rule 2	`(2, 3)`	`(2, 3)`	first dimension 2 vs 1: the 1 stretches to 2
Result	`(2, 3)`		shapes match, so the operation runs

4.2 Example 2: both arrays stretch

PYTHON • type this in Colab

```
a = np.arange(3).reshape((3, 1))
b = np.arange(3)
print(a.shape, b.shape)
print(a + b)
```

▶ OUTPUT

```
(3, 1) (3,)
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

Step	Shape of a	Shape of b	What happened
Start	`(3, 1)`	`(3,)`	
Rule 1	`(3, 1)`	`(1, 3)`	`b` padded on the left
Rule 2	`(3, 3)`	`(3, 3)`	`a`'s 1 stretches to 3, `b`'s 1 stretches to 3
Result	`(3, 3)`		a 3×3 grid

Here `.reshape((3, 1))` does the same job as `[:, np.newaxis]`: it turns a flat array into a column.

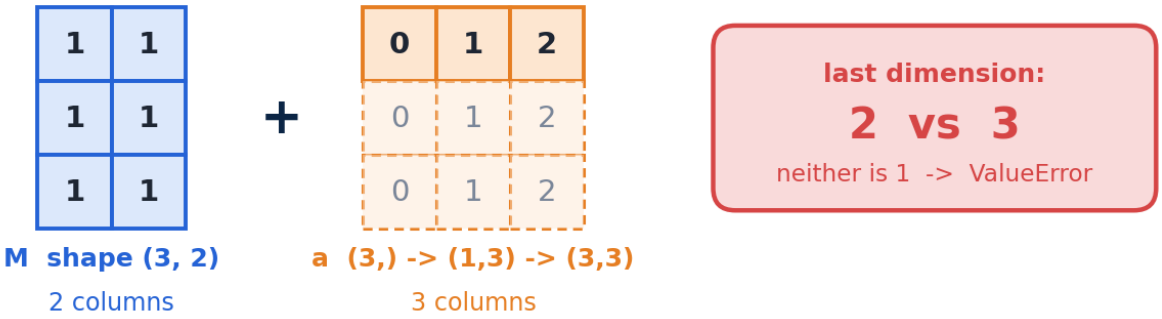
4.3 Example 3: the failure

PYTHON • type this in Colab

```
M = np.ones((3, 2))
a = np.arange(3)
print(M.shape, a.shape)
print(M + a)
```

▶ OUTPUT

```
(3, 2) (3,)
ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```



After padding, the last dimension is 2 against 3, and neither is 1.

Step	Shape of M	Shape of a	What happened
Start	`(3, 2)`	`(3,)`	
Rule 1	`(3, 2)`	`(1, 3)`	`a` padded on the left
Rule 3	`(3, 2)`	`(1, 3)`	last dimension: 2 vs 3, neither is 1: **ValueError**

INTERVIEW FLAG

"Why does `(3,2) + (3,)` fail even though both have a 3?"
Model answer: padding goes on the left, so the 3 of `(3,)` is lined up against the 2, not against the 3. The last dimension is 2 vs 3, neither is 1, so NumPy raises a ValueError.

4.4 How to read the error and fix it

The message names **both shapes that clashed**. When you see `could not be broadcast together`, do this:

TRY IT

Step 1. Print `.shape` of both arrays.
Step 2. Pad the shorter shape with 1s on the **left**, then compare from the **right**.
Step 3. Find the dimension where the sizes differ **and neither is 1**. That is your clash. Decide which axis you meant, and fix it with `np.newaxis` or `reshape`.

4.5 It works for every binary ufunc

We have used `+` as the example, but broadcasting applies to **any** operation that takes two arrays: `-`, `*`, `/`, `**`, `%`, comparisons, `np.maximum`, and so on.

PYTHON • type this in Colab

```
M = np.array([[1, 5, 2],
              [7, 0, 3]])
a = np.array([4, 4, 4])
print(np.maximum(M, a))
print(M * a)
print(M > a)
```

▶ OUTPUT

```
[[4 5 4]
 [7 4 4]]
[[ 4 20 8]
 [28 0 12]]
[[False True False]
 [ True False False]]
```

NOTE

The book also shows `np.logaddexp`, which computes `log(exp(a) + exp(b))` precisely, to prove the point. You do not need to learn that function; the lesson is only that the rules are the same for every binary ufunc.

✎ PRACTICE PROBLEMS | Part 4: The Rules on Real Code

Q1 **FIND THE BUGS**

`X` has shape (4, 3) and `w = np.array([1, 2, 3, 4])` holds **one weight per row**. Does `X * w` work as written? Show the padding and name the dimension that clashes. (Do not fix it yet; that is Part 5.)

PYTHON • type this in Colab

```
import numpy as np
X = np.ones((4, 3))
w = np.array([1, 2, 3, 4])
print(X * w)
```

Q2 **PREDICT THE OUTPUT**

Predict the shape and the output.

PYTHON • type this in Colab

```
import numpy as np
a = np.arange(3).reshape(3, 1)
b = np.arange(4)
print((a + b).shape)
print(a + b)
```

Q3 **READ & DECODE**

You see `ValueError: operands could not be broadcast together with shapes (5,2) (5,)`. Which dimension clashes, and why?

Q4 **PREDICT THE OUTPUT**

Predict both results.

PYTHON • type this in Colab

```
import numpy as np
M = np.array([[1, 5, 2], [7, 0, 3]])
a = np.array([4, 4, 4])
print(np.maximum(M, a))
print(M > a)
```

Q5 **THINK IT THROUGH**

Your friend says broadcasting only works with `+`. Give two other operations, with one example each.

Q6 **THEORY**

Describe the 3-step routine you follow when NumPy says `could not be broadcast together`. **[3 marks]**

5 Fixing a Failed Broadcast

PART 5

Block 5: np.newaxis, reshape and choosing the direction

IN THIS PART YOU WILL LEARN

- ✓ How to put the padding on the **right** when you need it
- ✓ The decision rule: does my 1D array line up with **rows** or **columns**?
- ✓ Why `reshape(-1, 1)` is everywhere in scikit-learn code
- ✓ The **silent bug** on square arrays: a wrong axis that gives no error

5.1 The problem

NumPy only pads on the **left**. If your 1D array should line up with the **rows** (one value per row), left-padding sends it the wrong way. Remember the weights from the last Part: 4 rows, 4 weights.

`X` has shape `(4, 3)` and `w` has 4 values, one per row. `X * w` fails: `(4,3)` against `(1,4)`, and 3 vs 4 clashes.

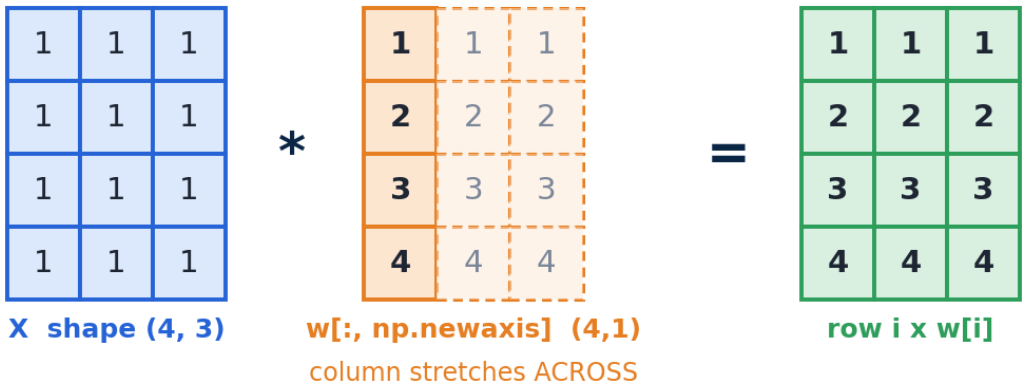
5.2 The fix: turn w into a column

PYTHON • type this in Colab

```
X = np.ones((4, 3))
w = np.array([1, 2, 3, 4])
print(w[:, np.newaxis].shape)
print(X * w[:, np.newaxis])
```

▶ OUTPUT

```
(4, 1)
[[1. 1. 1.]
 [2. 2. 2.]
 [3. 3. 3.]
 [4. 4. 4.]]
```



The column (4,1) stretches across 3 columns, so row i is multiplied by w[i].

Step	Shape of X	Shape of w	What happened
Start	`(4, 3)`	`(4, 1)`	we made `w` a column ourselves
Dimension 1	4	4	equal: passes
Dimension 2	3	1	the 1 stretches to 3
Result	`(4, 3)`		row `i` of X multiplied by `w[i]`

5.3 Two ways to make a column

PYTHON • type this in Colab

```
w = np.array([1, 2, 3, 4])
print(w[:, np.newaxis].shape)
print(w.reshape(4, 1).shape)
print(w.reshape(-1, 1).shape)
```

▶ OUTPUT

```
(4, 1)
(4, 1)
(4, 1)
```

`w.reshape(-1, 1)` is very common in real code. The `-1` means "figure this dimension out yourself", so it works for any length. You will see it constantly in scikit-learn.

5.4 The decision rule

Ask yourself: **which axis of the bigger array should my small array line up with?**

I want my 1D array of size n to...	Use	Shape
line up with the **columns** (one value per column)	nothing needed, or `a[np.newaxis, :]`	`(1, n)`
line up with the **rows** (one value per row)	`a[:, np.newaxis]` or `a.reshape(-1, 1)`	`(n, 1)`

5.5 The silent trap on square arrays

On a square array both directions have the same length, so the wrong choice runs **without an error**:

PYTHON • type this in Colab

```
X = np.arange(9).reshape(3, 3)
a = np.array([10, 20, 30])
print(X)
print(X + a) # a lines up with COLUMNS
print(X + a[:, np.newaxis]) # a lines up with ROWS
```

▶ OUTPUT

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[10 21 32]
 [13 24 35]
 [16 27 38]]
[[10 11 12]
 [23 24 25]
 [36 37 38]]
```

COMMON TRAP

On a square array, the wrong axis does not crash; it silently gives a wrong answer. `X + a` adds `[10, 20, 30]` to every **row** (so each **column** gets one value). `X + a[:, np.newaxis]` adds 10 to row 0, 20 to row 1, 30 to row 2. Always check which axis you meant.

✎ PRACTICE PROBLEMS | Part 5: Fixing a Failed Broadcast

Q1 FIND THE BUGS

`M` has shape (2, 3) and `a = np.array([10, 20])`. (1) Does `M + a` work? Show the clash. (2) Write the code that adds 10 to the first row and 20 to the second row. (3) Give the output.

PYTHON • type this in Colab

```
import numpy as np
M = np.array([[1, 2, 3],
              [4, 5, 6]])
a = np.array([10, 20])
```

Q2 BUILD IT

`X` has shape (3, 2) and `w = np.array([2, 3, 4])` is one scale factor per row. Write one line that multiplies each row of `X` by its factor.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1, 1], [2, 2], [3, 3]])
w = np.array([2, 3, 4])
```

Q3 PREDICT THE OUTPUT

Predict both shapes, then the two outputs. Say which axis each version lines up with.

PYTHON • type this in Colab

```
import numpy as np
X = np.zeros((3, 3))
b = np.array([1, 2, 3])
print(X + b)
print(X + b.reshape(-1, 1))
```

Q4 THINK IT THROUGH

Why is the square-array case more dangerous than a (4, 3) array with a length-4 vector?

Q5 CALCULATE

What is the shape of `np.arange(6).reshape(-1, 1)`, and of `np.arange(6).reshape(-1, 2)`?

Q6 THEORY

State the decision rule for choosing between `a` and `a[:, np.newaxis]` when broadcasting a 1D array against a 2D array. **[3 marks]**

PART 6

6 Broadcasting in Real Use

Block 5: centring data, keepdims and the 2D function grid

IN THIS PART YOU WILL LEARN

- ✓ Centre every **column** of a data table (subtract the feature means)
- ✓ Centre every **row**, and why `keepdims=True` is needed
- ✓ Scale columns by their maximum
- ✓ Evaluate a function over a whole 2D grid with a row and a column
- ✓ Read the broadcasting inside the standardisation line from Block 4

6.1 Centring each column

This is the first step of normalisation and PCA. By convention ML data is **samples as rows, features as columns**. Here 10 samples with 3 features:

PYTHON • type this in Colab

```
rng = np.random.default_rng(0)
X = rng.random((10, 3))
Xmean = X.mean(axis=0)
print(Xmean.shape)
X_centered = X - Xmean
print(np.round(X_centered.mean(axis=0), 12) + 0.0)
print(np.allclose(X_centered.mean(axis=0), 0))
```

▶ OUTPUT

```
(3,)
[0. 0. 0.]
True
```

Object	Shape	Why
`X`	`(10, 3)`	10 samples, 3 features
`X.mean(axis=0)`	`(3,)`	axis 0 (the rows) collapses: one mean per column
`Xmean` after Rule 1	`(1, 3)`	padded on the left; stretches down to 10 rows
`X - Xmean`	`(10, 3)`	each column has its own mean subtracted

IMPORTANT The centred means print as tiny numbers like `2e-17` instead of exactly 0 because of floating-point precision. Never test with `== 0`; use `np.allclose(..., 0)` (or `np.isclose`).

Notice that no `np.newaxis` was needed. The mean is one value per column, so left-padding already lines it up correctly.

6.2 Centring each row: the silent bug and keepdims

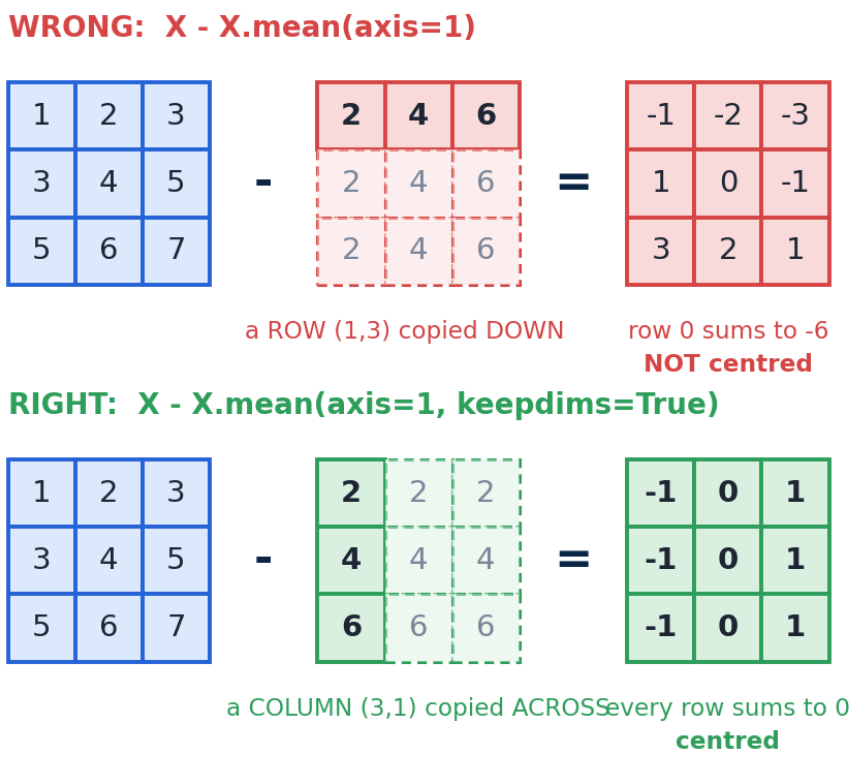
Now you want every **row** to have mean 0 (subtract each sample's own mean). `X.mean(axis=1)` has one value **per row**, so it needs to be a **column**. The tempting version is wrong:

PYTHON • type this in Colab

```
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
print(X.mean(axis=1))
print(X - X.mean(axis=1)) # WRONG
print(X - X.mean(axis=1, keepdims=True)) # RIGHT
```

▶ OUTPUT

```
[2. 4. 6.]
[[-1. -2. -3.]
 [ 1. 0. -1.]
 [ 3. 2. 1.]]
[[-1. 0. 1.]
 [-1. 0. 1.]
 [-1. 0. 1.]]
```



Wrong: the row [2 4 6] is copied down. Right: the column [2; 4; 6] is copied across.

The wrong version runs, because `(3,)` pads to `(1, 3)`, which is compatible with a `(3, 3)` array. But it subtracts 2 from column 0, 4 from column 1 and 6 from column 2: the wrong axis. A **non-square** array would have raised an error and warned you; the square array hides it.

MEMORY TRICK

After an aggregation along an axis, use `keepdims=True` if you plan to broadcast the result back against the original array. `keepdims=True` keeps the collapsed axis as size 1, so a `(3, 3)` array averaged along axis 1 gives `(3, 1)` instead of `(3,)`.

PYTHON • type this in Colab

```
print(X.mean(axis=1).shape)
print(X.mean(axis=1, keepdims=True).shape)
print(X.mean(axis=1)[:, np.newaxis].shape)
```

▶ OUTPUT

```
(3,)
(3, 1)
(3, 1)
```

WHY IT MATTERS

"Compute a statistic along one axis, then broadcast it back" is the shape of **softmax**, **layer normalisation** and **attention**. Those three are the core of every language model, and all three rely on `keepdims=True` or its equivalent.

6.3 Scaling each column by its maximum

PYTHON • type this in Colab

```
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
print(X.max(axis=0))
print(X / X.max(axis=0))
```

▶ OUTPUT

```
[5. 6. 7.]
[[0.2 0.33333333 0.42857143]
 [0.6 0.66666667 0.71428571]
 [1. 1. 1. ]]
```

No `keepdims` is needed here. `X.max(axis=0)` has shape `(3,)`, which pads to `(1, 3)` and stretches down, so column `j` is divided by its own maximum. Axis 0 aggregation leaves exactly the shape that left-padding expects.

6.4 A function over a whole grid

To evaluate `z = f(x, y)` for every combination of `x` and `y` without nested loops, make `x` a row and `y` a column. This is the same row-plus-column grid trick as Part 2.

PYTHON • type this in Colab

```
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 50)[:, np.newaxis]
z = np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
print(x.shape, y.shape, z.shape)
```

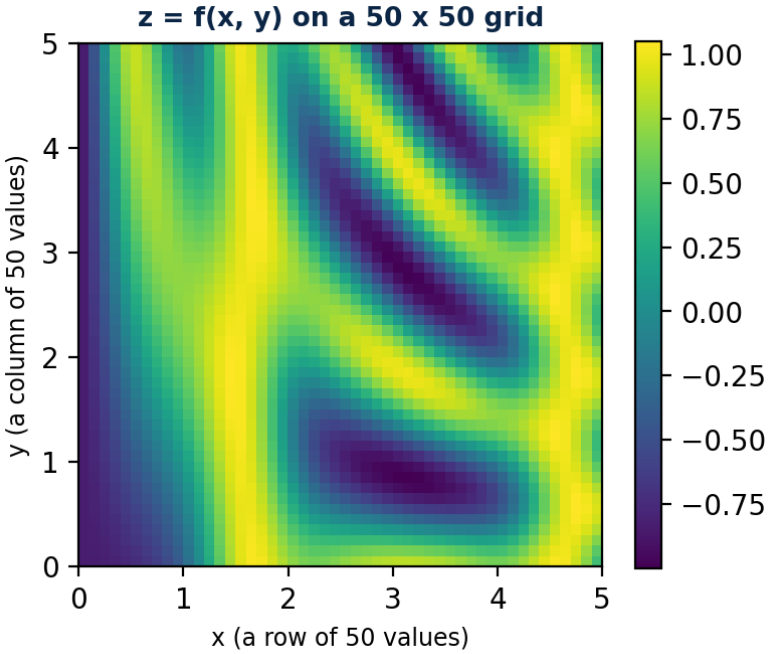
▶ OUTPUT

```
(50,) (50, 1) (50, 50)
```

`np.linspace(0, 5, 50)` gives **50 evenly spaced numbers from 0 to 5, both ends included**. Then `y * x` is `(50, 1) * (50,)`, which broadcasts to `(50, 50)`. Every operation on `x` and `y` after that works on the full grid, so `z[i, j]` is the value of the function at `y[i]` and `x[j]`.

PYTHON • type this in Colab

```
import matplotlib.pyplot as plt
plt.imshow(z, origin='lower', extent=[0, 5, 0, 5], cmap='viridis')
plt.colorbar()
```



The picture from the book. Each pixel is one cell of the 50 x 50 grid and the colour is the value of z.

NOTE

You do **not** need to understand this particular formula. You only need to understand **why a row `x` and a column `y` produce a grid**. The picture is just `plt.imshow` colouring the 50 x 50 array.

WHY IT MATTERS

Loss surfaces, decision boundaries and heatmaps are all computed on a grid like this. `np.meshgrid` does the same job with explicit arrays, and you will meet it later.

6.5 The standardisation line, decoded

In Block 4 you wrote `Z = (X - X.mean(axis=0)) / X.std(axis=0)` and were told it was broadcasting. Now you can read it fully:

PYTHON • type this in Colab

```
X = np.array([[170, 65], [160, 50], [180, 80], [175, 75]])
print(X.mean(axis=0).shape, X.std(axis=0).shape)
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(np.round(Z, 2))
```

▶ OUTPUT

```
(2,) (2,)
[[-0.17 -0.22]
 [-1.52 -1.53]
 [ 1.18 1.09]
 [ 0.51 0.65]]
```

Both `X.mean(axis=0)` and `X.std(axis=0)` have shape `(2,)`. They pad to `(1, 2)` and stretch down 4 rows, so each **feature** (column) is centred by its own mean and divided by its own standard deviation.

✎ PRACTICE PROBLEMS | Part 6: Broadcasting in Real Use

Q1 **CALCULATE**

Let `X = [[1,2,3],[3,4,5],[5,6,7]]` (floats). (1) Compute the mean of each column and give its shape. (2) Write the code to centre each column and give the output. (3) Why is `X - X.mean(axis=1)` wrong for centring each **row**, even though it raises no error? How do you fix it?

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
```

Q2 **BUILD IT**

Divide each **column** of the same `X` by its maximum. Do you need `keepdims=True` or `np.newaxis`? Why or why not?

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
```

Q3 **BUILD IT**

Make every **row** of `X` sum to 1 (divide each row by its own total). This is the pattern behind softmax.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1., 2., 1.],
              [2., 2., 4.],
              [5., 5., 0.]])
```

Q4 **FIND THE BUGS**

`X` has shape (4, 3). A student writes `X - X.mean(axis=1)` to centre each row and gets a ValueError. Why, and what is the fix?

PYTHON • type this in Colab

```
import numpy as np
X = np.arange(12.).reshape(4, 3)
print(X - X.mean(axis=1))
```

Q5 **CALCULATE**

`x = np.linspace(0, 1, 50)` and `y = np.linspace(0, 1, 30)[:, np.newaxis]`. What is the shape of `x * y`, and what does `z[i, j]` refer to?

Q6 **READ & DECODE**

Explain in plain words what `(X - X.mean(axis=0)) / X.std(axis=0)` does for `X` of shape (samples, features), and name every shape involved.

Q7 **THEORY**

Explain `keepdims=True` with an example and say why it helps broadcasting. **[4 marks]**

PART 7

7 Comparisons Give Boolean Arrays

Block 6, first sections: comparison operators as ufuncs

IN THIS PART YOU WILL LEARN

- ✓ Why we need masks: answering questions about data without writing loops
- ✓ The six comparison operators and the Boolean array they return
- ✓ Compound expressions, 2D arrays and broadcasting inside a comparison
- ✓ The ufunc names hiding behind `<`, `>`, `==` and friends
- ✓ How `np.mean(preds == y)` becomes an accuracy score
- ✓ Why you must never test floats with `==`

7.1 The problem masks solve

Block 6 starts with a data question. Imagine an array with the rainfall of every day of a year (365 numbers). You want to know: **how many days were rainy? What was the average rain on rainy days? How many days had more than half an inch?** A histogram gives a rough picture but cannot answer these exactly.

One way is a Python loop with a counter. You already know from Block 3 why that is slow and clumsy. NumPy's answer is the same as before: use a **ufunc** that works on the whole array at once. This time the ufuncs are the **comparison operators**, and the result is a **mask**.

ANALOGY

The attendance register. A teacher does not walk to every student and count the ones taller than 160 cm. She holds a sheet with one tick or cross per student. Ticks and crosses are easy to count, to combine and to use for picking students. A **Boolean array** is that sheet: one True or False for every element.

7.2 The six comparison operators

In Block 3 you saw that `+ - * /` work element by element. The comparison operators do the same, but the answer for every element is **True or False**:

PYTHON • type this in Colab

```
x = np.array([1, 2, 3, 4, 5])
print(x < 3)
print(x > 3)
print(x <= 3)
print(x >= 3)
print(x != 3)
print(x == 3)
```

▶ OUTPUT

```
[ True True False False False]
[False False False True True]
[ True True True False False]
[False False True True True]
[ True True False True True]
[False False True False False]
```

Read one result against the input, position by position:

x	1	2	3	4	5
x < 3	True	True	False	False	False
x == 3	False	False	True	False	False

MEMORY TRICK

A comparison is a question asked to every element. `x < 3` asks "is this one less than 3?" five separate times and writes down five answers. The answer array always has the **same shape** as the input, and its dtype is **bool**.

COMMON TRAP

`==` **compares,** `=` **assigns.** `x == 3` gives a Boolean array and leaves `x` alone. `x = 3` throws your array away and stores the number 3. Mixing them up is the most common typing slip in this chapter.

7.3 Compound expressions

Each side of a comparison can be a full expression. NumPy computes both sides first (as arrays), then compares them element by element:

PYTHON • type this in Colab

```
x = np.array([1, 2, 3, 4, 5])
print((2 * x) == (x ** 2))
```

▶ OUTPUT

```
[False True False False False]
```

Only position 1 is True: there `2 * 2 = 4` and `2 ** 2 = 4`. Everywhere else the two sides differ. (Position 0 gives `2` against `1`, so False.)

7.4 Any shape, and broadcasting works too

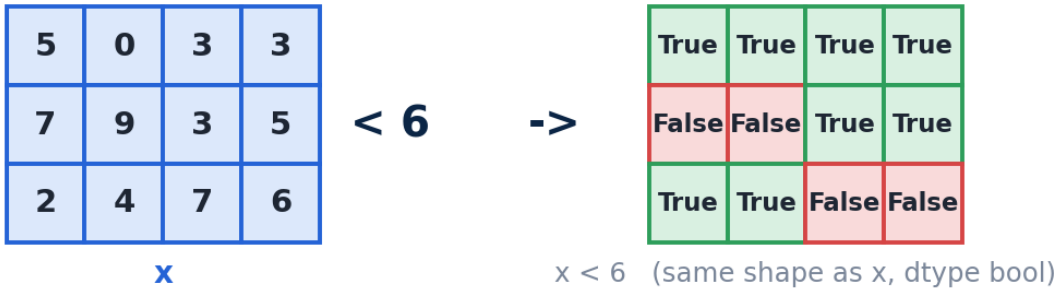
Comparisons work on arrays of any shape. The answer has the same shape as the input:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(x < 6)
```

▶ OUTPUT

```
[[ True True True True]
 [False False True True]
 [ True True False False]]
```



The 6 was stretched (broadcast) to a 3 x 4 array, then every element was compared with it. Result: a 3 x 4 Boolean array.

Because the comparison is a binary ufunc, **all the broadcasting rules of Block 5 apply**. You can compare a 2D array with a 1D array of thresholds, one threshold per column:

PYTHON • type this in Colab

```
x = np.arange(12).reshape(3, 4)
t = np.array([2, 5, 8, 11]) # one threshold per column
print(x)
print(x > t)
```

▶ OUTPUT

```
[[ 0 1 2 3]
 [ 4 5 6 7]
 [ 8 9 10 11]]
[[False False False False]
 [ True False False False]
 [ True True True False]]
```

What got stretched? `t` has shape `(4,)`, which pads to `(1, 4)` and stretches down 3 rows, so column `j` is compared with `t[j]`.

7.5 The ufuncs hiding behind the operators

Exactly as `+` is a wrapper for `np.add`, each comparison operator is a wrapper for a ufunc. `x < 3` is really `np.less(x, 3)`. You will almost never type these names. Know that they exist **LOOK UP**:

Operator	Equivalent ufunc	Operator	Equivalent ufunc
`==`	`np.equal`	`!=`	`np.not_equal`
`<`	`np.less`	`<=`	`np.less_equal`
`>`	`np.greater`	`>=`	`np.greater_equal`

PYTHON • type this in Colab

```
x = np.array([1, 2, 3, 4, 5])
print(np.less(x, 3))
print(np.array_equal(np.less(x, 3), x < 3))
```

▶ OUTPUT

```
[ True True False False False]
True
```

7.6 The payoff: accuracy in one line

A classifier predicts labels `preds`. The true labels are `y`. Comparing them gives one True or False per sample. True means the prediction was right:

PYTHON • type this in Colab

```
preds = np.array([1, 0, 1, 1])
y = np.array([1, 1, 1, 0])
print(preds == y)
print(np.mean(preds == y))
```

▶ OUTPUT

```
[ True False True False]
0.5
```

The mean of the Booleans is the fraction that are True, because True counts as 1 and False as 0 (Part 8 explains this fully). Two of four were correct, so the accuracy is 0.5.

WHY IT MATTERS

`np.mean(preds == y)` is the accuracy of a model. `probs > 0.5` turns predicted probabilities into class labels. `X > 0` is the heart of the ReLU activation. Filtering data, finding mistakes and thresholding all start with a comparison.

7.7 Never compare floats with ==

Computers store decimals approximately (you met this with `np.sin(np.pi)` in Block 3). So two numbers that are equal on paper may differ in the last digit:

PYTHON • type this in Colab

```
print(0.1 + 0.2 == 0.3)
print(0.1 + 0.2)
print(np.isclose(0.1 + 0.2, 0.3))
```

▶ OUTPUT

```
False
0.30000000000000004
True
```

COMMON TRAP

`==` **on floats is unreliable.** `0.1 + 0.2` is `0.30000000000000004`, so `== 0.3` says False. Use `np.isclose(a, b)` for floats **MUST KNOW**. `==` is safe for integers and labels, which is what you use for accuracy.

✎ PRACTICE PROBLEMS | Part 7: Comparisons Give Boolean Arrays

Q1 **PREDICT THE OUTPUT**

Predict all three lines.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([4, 8, 15, 16, 23, 42])
print(x > 15)
print(x % 2 == 0)
print(x != 16)
```

Q2 **PREDICT THE OUTPUT**

Predict both lines. What does the second one measure?

PYTHON • type this in Colab

```
import numpy as np
preds = np.array([1, 0, 1, 1])
y = np.array([1, 1, 1, 0])
print(preds == y)
print(np.mean(preds == y))
```

Q3 **PREDICT THE OUTPUT**

Predict the output. What got stretched, and in which direction?

PYTHON • type this in Colab

```
import numpy as np
x = np.arange(12).reshape(3, 4)
t = np.array([2, 5, 8, 11])
print(x > t)
```

Q4 **READ & DECODE**

Explain in plain words what the second line does and what `labels` contains.

PYTHON • type this in Colab

```
import numpy as np
probs = np.array([0.2, 0.7, 0.5, 0.9])
labels = probs > 0.5
print(labels)
```

Q5 **FIND THE BUGS**

A student wants to check whether the sum of two floats is 0.3. The check says False even though on paper it is 0.3. Explain and fix it.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([0.1, 0.2, 0.3])
print(a[0] + a[1] == a[2])
```

Q6 **CALCULATE**

For `m = np.arange(20).reshape(4, 5) >= 10` give (a) the shape of `m`, (b) its dtype, (c) how many True values there are.

Q7 **THEORY**

Explain in three or four lines why `x < 3` is called a ufunc operation. Name the function that `x < 3` calls. **[3 marks]**

PART 8

8 Counting and Checking

Block 6: np.sum, np.count_nonzero, np.any and np.all on Boolean arrays

IN THIS PART YOU WILL LEARN

- ✓ How to count the True values with `np.sum` and `np.count_nonzero`
- ✓ Why `np.sum` and `np.mean` work on Booleans (True is 1, False is 0)
- ✓ Counting per row and per column with `axis`
- ✓ `np.any` (at least one) and `np.all` (every single one)
- ✓ The built-in `sum`, `any`, `all` trap on 2D arrays

8.1 Counting the True values

A Boolean array rarely ends the story. Usually you want a **number** ("how many?") or a **yes/no** ("is there any?"). Use this array for the whole Part:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(x)
```

▶ OUTPUT

```
[[5 0 3 3]
 [7 9 3 5]
 [2 4 7 6]]
```

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(np.count_nonzero(x < 6))
print(np.sum(x < 6))
```

▶ OUTPUT

```
8
8
```

Both answer: **how many values are less than 6?** `np.count_nonzero` counts the non-zero entries, and a True counts as non-zero. `np.sum` works because NumPy treats **True as 1 and False as 0**, so adding them counts the Trues.

MEMORY TRICK

True = 1, False = 0. Once you remember this, counting is `np.sum(mask)` and the fraction is `np.mean(mask)`. Both are **MUST KNOW**.

8.2 Counting per row and per column

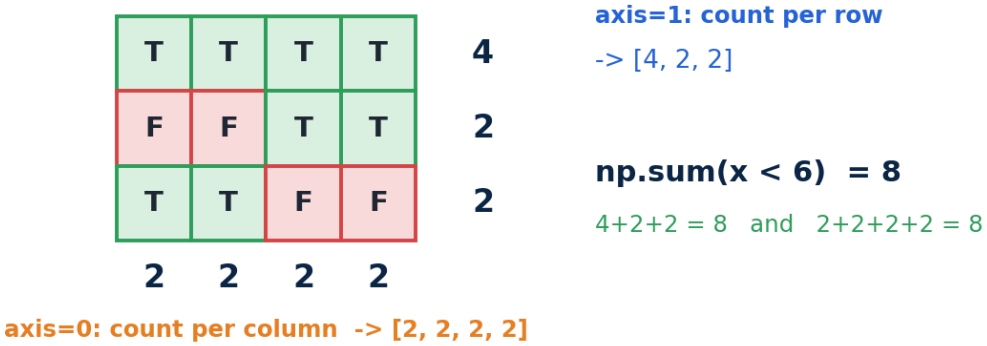
The big advantage of `np.sum` is that it takes the `axis` argument you learned in Block 4:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(np.sum(x < 6, axis=1)) # per row
print(np.sum(x < 6, axis=0)) # per column
```

▶ OUTPUT

```
[4 2 2]
[2 2 2 2]
```



axis=1 collapses the columns, so you get one count per row. axis=0 collapses the rows, so you get one count per column. Both add up to the same total, 8.

IMPORTANT **Sanity check on every count:** the row counts and the column counts must each add up to the grand total. Here `4 + 2 + 2 = 8` and `2 + 2 + 2 + 2 = 8`.

8.3 The fraction of True values

Because True is 1 and False is 0, the **mean** of a Boolean array is the **fraction** that are True:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(np.mean(x < 6))
print(np.mean(x < 6, axis=1))
```

▶ OUTPUT

```
0.6666666666666666
[1. 0.5 0.5]
```

8 of the 12 values are below 6 (8 / 12 = 0.667). Row by row the fractions are 4/4, 2/4 and 2/4. This is exactly how accuracy works: `np.mean(preds == y)`.

8.4 np.any and np.all

Two functions give a single yes/no answer about a Boolean array:

Function	True when	Think of it as
`np.any(mask)`	at least one element is True	"is there at least one...?"
`np.all(mask)`	every single element is True	"are they all...?"

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(np.any(x > 8)) # is any value above 8?
print(np.any(x < 0)) # is any value negative?
print(np.all(x < 10)) # are all values below 10?
print(np.all(x == 6)) # are all values equal to 6?
```

▶ OUTPUT

```
True
False
True
False
```

Both accept `axis`, so you can ask the question for each row or column:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(np.all(x < 8, axis=1))
```

▶ OUTPUT

```
[ True False True]
```

The middle row `[7, 9, 3, 5]` contains a 9, so not all of its values are below 8. The other two rows pass.

MEMORY TRICK `any` = at least one, `all` = every single one.

8.5 A useful ML check: is there missing data?

Missing values appear as `np.nan`. `np.isnan(X)` is a comparison-style function that returns a Boolean array, so the tools above apply directly:

PYTHON • type this in Colab

```
X = np.array([[1.0, 2.0],
              [3.0, np.nan],
              [5.0, 6.0]])
print(np.isnan(X))
print(np.any(np.isnan(X)))
print(np.sum(np.isnan(X), axis=0))
```

▶ OUTPUT

```
[[False False]
 [False True]
 [False False]]
True
[0 1]
```

WHY IT MATTERS `np.any(np.isnan(X))` is the very first sanity check on a new dataset: does it contain missing values? The per-column count tells you which features need cleaning before training.

8.6 The built-in sum, any and all trap

Python itself has built-in `sum()`, `any()` and `all()`. They look the same but are designed for plain lists. On a 2D NumPy array they behave differently:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(sum(x < 6)) # built-in sum
print(np.sum(x < 6)) # NumPy sum
```

▶ OUTPUT

```
[2 2 2 2]
8
```

COMMON TRAP **The built-in `sum` on a 2D array does not raise an error. It silently gives the wrong kind of answer.** It adds up the rows one after another, so you get an array of 4 column counts instead of the single total 8. The built-in `any` and `all` can fail with a ValueError. Rule: on arrays always write `np.sum`, `np.any`, `np.all`.

✎ PRACTICE PROBLEMS | Part 8: Counting and Checking

Q1 **PREDICT THE OUTPUT**

Predict all four values.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
print(np.sum(x > 4))
print(np.sum(x > 4, axis=1))
print(np.all(x == 4, axis=1))
print(np.any(x == 4))
```

Q2 **PREDICT THE OUTPUT**

Predict both lines. Why is the second one a number between 0 and 1?

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
print(np.mean(x > 4))
print(np.mean(x > 4, axis=0))
```

Q3 **BUILD IT**

For the same `x`, write code that (a) counts the values greater than 3 in **each column**, and (b) tells whether **any row** has all its values greater than 3.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
```

Q4 **FIND THE BUGS**

A student counts the values above 4 and expects the number 4. The output is not a single number. Explain and fix.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
print(sum(x > 4))
```

Q5 **BUILD IT**

Given the array below, write code that (a) tells whether the data has any NaN, and (b) counts the NaNs in each column.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1.0, np.nan, 3.0],
              [4.0, 5.0, np.nan],
              [np.nan, np.nan, 9.0]])
```

Q6 **CALCULATE**

A mask `m` has shape (6, 3). Give the shape of `np.sum(m, axis=0)`, `np.sum(m, axis=1)` and `np.sum(m)`.

Q7 **THINK IT THROUGH**

Why does `np.sum(x < 6)` count the values below 6? What exactly is being added?

Q8 **THEORY**

Explain the difference between `np.any` and `np.all` with one example each, and say what the `axis` argument does for them. **[4 marks]**

PART 9

9 Combining Conditions

Block 6: &, | and ~ on Boolean arrays, and why brackets are mandatory

IN THIS PART YOU WILL LEARN

- ✓ How to ask "between", "either/or" and "not" questions with &, |, ~
- ✓ Why every comparison needs its own brackets (operator precedence)
- ✓ De Morgan's law: two ways to write the same mask
- ✓ The ~ trap on non-Boolean integer arrays
- ✓ The bitwise ufuncs hiding behind the operators

9.1 Why one condition is not enough

Real questions have several conditions: rain **between** 0.5 and 1 inch, values that are **either** very small **or** very large, everything **except** the outliers. Combine Boolean arrays with three operators:

Meaning	Operator	Example	True when
AND	&	`(x > 2) & (x < 5)`	both conditions are True
OR	|	`(x < 2) | (x > 5)`	at least one is True
NOT	~	`~(x > 3)`	the condition is False
XOR	^	`(a) ^ (b)`	exactly one is True (rare) **LOOK UP**

Like every other ufunc, these three work **element by element** on arrays.

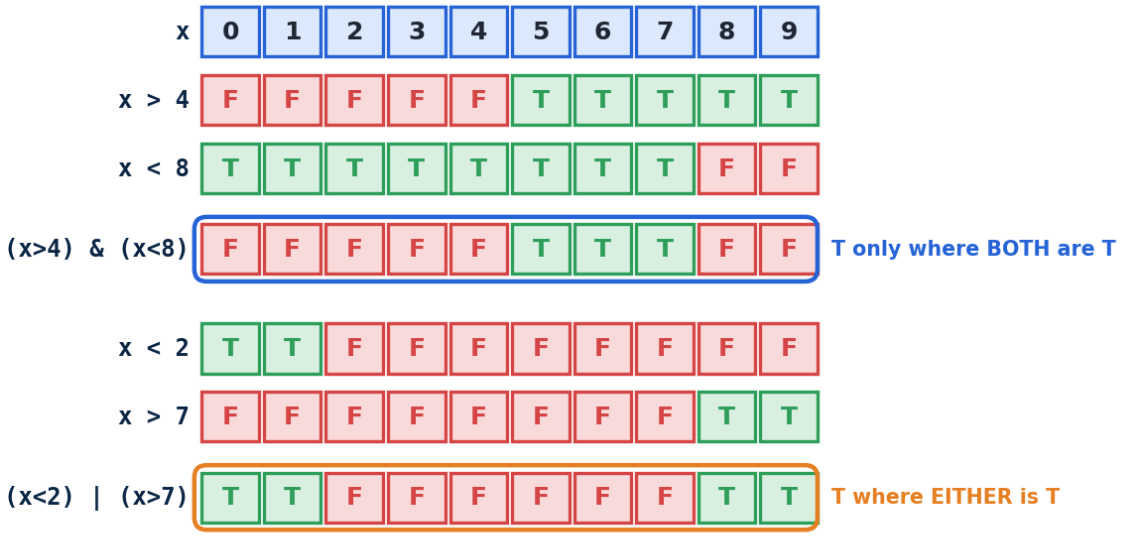
9.2 AND, OR, NOT in action

PYTHON • type this in Colab

```
x = np.arange(10)
print(x)
print((x > 4) & (x < 8))
print((x < 2) | (x > 7))
```

▶ OUTPUT

```
[0 1 2 3 4 5 6 7 8 9]
[False False False False False True True True False False]
[ True True False False False False False False True True]
```



Position by position: AND is True only where both rows are True. OR is True where at least one row is True.

Combine with the counting tools of Part 8:

PYTHON • type this in Colab

```
x = np.arange(10)
print(np.sum((x > 4) & (x < 8))) # 5, 6, 7
print(np.sum((x < 2) | (x > 7))) # 0, 1, 8, 9
print(np.sum(~(x > 3))) # 0, 1, 2, 3
```

▶ OUTPUT

```
3
4
4
```

MEMORY TRICK `&` is "and", `|` is "or", `~` is "not". Say it while typing: **every comparison gets its own brackets.**

`~(x > 3)` and `x <= 3` give the same mask. Use whichever reads more clearly.

9.3 The brackets are mandatory

This is the most common error in this whole block:

PYTHON • type this in Colab

```
x = np.arange(10)
print((x > 4) & (x < 8)) # correct
print(x > 4 & x < 8) # no brackets
```

▶ OUTPUT

```
[False False False False False True True True False False]
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or
a.all()
```

The reason is **operator precedence**: in Python, `&` binds **tighter** than `>` and `<`. So without brackets the line is read as `x > (4 & x) < 8`, which is nonsense. The brackets force each comparison to run first.

IMPORTANT

Rule: whenever you use &, | or ~, wrap every comparison in its own brackets. `(x > 4) & (x < 8)` is correct. `x > 4 & x < 8` is wrong, always.

9.4 De Morgan's law (two ways to say the same thing)

"A and B" is the same as "not (not A or not B)". You may remember this from a logic course. It explains why two masks that look different are equal:

PYTHON • type this in Colab

```
x = np.arange(10)
m1 = (x > 4) & (x < 8)
m2 = ~((x <= 4) | (x >= 8))
print(np.sum(m1), np.sum(m2))
print(np.array_equal(m1, m2))
```

▶ OUTPUT

```
3 3
True
```

You do not need to write masks this way. The law only helps you read someone else's code and see that two forms agree.

9.5 The ~ trap on integer arrays

~ means NOT only for **Boolean** arrays. On an integer array it flips the **bits** of each number, which gives strange results:

PYTHON • type this in Colab

```
flags = np.array([1, 0, 1])
print(~flags)
print(~flags.astype(bool))
```

▶ OUTPUT

```
[-2 -1 -2]
[False True False]
```

COMMON TRAP

~ on a 0/1 integer array does not give 0/1. It gives `-2` and `-1`, because it flips every bit. Apply ~ only to real Boolean masks (the output of a comparison), or convert first with `.astype(bool)`.

9.6 The ufuncs behind the operators

Operator	Equivalent ufunc	Operator	Equivalent ufunc
&	np.bitwise_and	|	np.bitwise_or
^	np.bitwise_xor	~	np.bitwise_not

WHY IT MATTERS

Filtering a table with several conditions (`(age > 18) & (income < 50000)`), removing outliers with `(x < low) | (x > high)` and building attention masks in transformers all use &, | and ~. Pandas filtering uses exactly the same syntax.

✎ PRACTICE PROBLEMS | Part 9: Combining Conditions

Q1 **PREDICT THE OUTPUT**

Predict all three lines.

PYTHON • type this in Colab

```
import numpy as np
x = np.arange(10)
print((x > 4) & (x < 8))
print(np.sum((x < 2) | (x > 7)))
print(np.sum(~(x > 3)))
```

Q2 **PREDICT THE OUTPUT**

Predict the output. Then write the code that counts the values **not** between 5 and 10 (both inclusive) using `~`, and give the count.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([3, 7, 12, 5, 20, 9, 1])
print((x > 4) & (x < 10))
```

Q3 **FIND THE BUGS**

This line is meant to count the values strictly between 4 and 10 but it fails. Explain why in one sentence and fix it.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([3, 7, 12, 5, 20, 9, 1])
print(np.sum(x > 4 & x < 10))
```

Q4 **BUILD IT**

For `x = [10, -3, 25, 8, -12, 40, 0, 17]` count (a) the values outside the range 0 to 20 (inclusive), and (b) the values that are even **and** positive.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([10, -3, 25, 8, -12, 40, 0, 17])
```

Q5 **THINK IT THROUGH**

Show with code that `(x > 4) & (x < 8)` and `~((x <= 4) | (x >= 8))` give the same mask for `x = np.arange(10)`. Which law is this?

Q6 **FIND THE BUGS**

A student has a 0/1 integer array `flags` and wants the opposite flags (1 becomes 0, 0 becomes 1). What goes wrong, and how do you fix it?

PYTHON • type this in Colab

```
import numpy as np
flags = np.array([1, 0, 1])
print(~flags)
```

Q7 **READ & DECODE**

Read this line and explain in plain words what the mask selects. Name the shape of `mask` for a `(100, 2)` array `X`.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1, 3], [-2, 8], [4, 5], [6, 1]])
mask = (X[:, 0] > 0) & (X[:, 1] <= 5)
print(mask)
```

Q8 **THEORY**

Why do `&`, `|` and `~` need brackets around each comparison? Give one correct and one wrong line. **[3 marks]**

PART 10

10 Boolean Masks: Selecting Values

Block 6: x[mask], modifying through a mask, and the rainfall example

IN THIS PART YOU WILL LEARN

- ✓ How to select values with `x[mask]` and why the result is always 1D
- ✓ Computing statistics on the selected values
- ✓ Storing masks in named variables and combining them
- ✓ Selecting whole rows with a 1D mask (`X[y == 1]`)
- ✓ Changing values through a mask (`x[x < 0] = 0`)
- ✓ Why `x[mask]` makes a copy, and what a wrong-length mask does

10.1 From counting to selecting

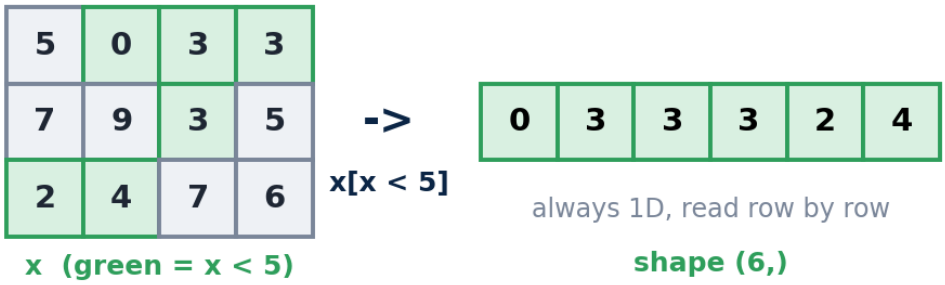
So far a Boolean array gave you counts and yes/no answers. The more powerful idea is to use it as a **mask** to pull out the values themselves. Put the Boolean array inside the square brackets:

PYTHON • type this in Colab

```
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(x < 5)
print(x[x < 5])
```

▶ OUTPUT

```
[[False True True True]
 [False False True False]
 [ True True False False]]
[0 3 3 3 2 4]
```



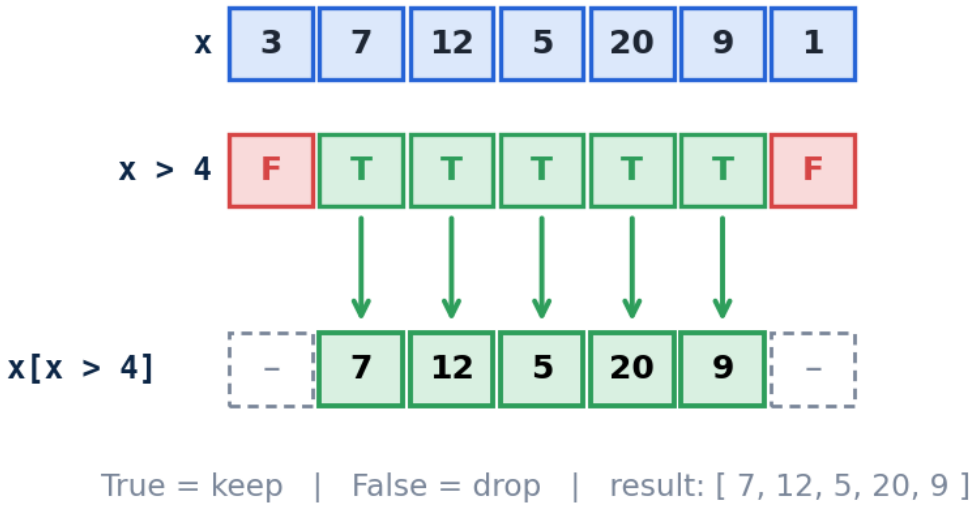
The mask keeps the green cells. The result is always a flat 1D array, read row by row.

ANALOGY

A stencil. Lay a stencil with holes over a page. You only see the words under the holes. True means "hole", False means "covered". `x[mask]` gives you only the values you can see through the stencil.

IMPORTANT **The result is always 1D**, even when x is 2D, because scattered elements cannot form a rectangle. The values come out in row-by-row order. The mask must have the same shape as the array it indexes.

10.2 Selecting from a 1D array



True = keep, False = drop. The order of the kept values never changes.

PYTHON • type this in Colab

```
x = np.array([3, 7, 12, 5, 20, 9, 1])
print(x[x > 4])
print(x[(x > 4) & (x < 10)])
print(x[(x < 4) | (x > 15)])
```

▶ OUTPUT

```
[ 7 12 5 20 9]
[7 5 9]
[ 3 20 1]
```

10.3 Statistics on the selected values

Once you have the selected values you can do anything with them: this is how you ask "what is the mean of just these?"

PYTHON • type this in Colab

```
x = np.array([3, 7, 12, 5, 20, 9, 1])
print(x[x > 4].mean())
print(np.max(x[x < 10]))
print(np.median(x[x > 4]))
```

▶ OUTPUT

```
10.6
9
9.0
```

MEMORY TRICK

Counting uses `np.sum(mask)`. Selecting uses `x[mask]`. The same mask feeds both. One answers "how many?", the other answers "which ones?".

10.4 Named masks and the rainfall example

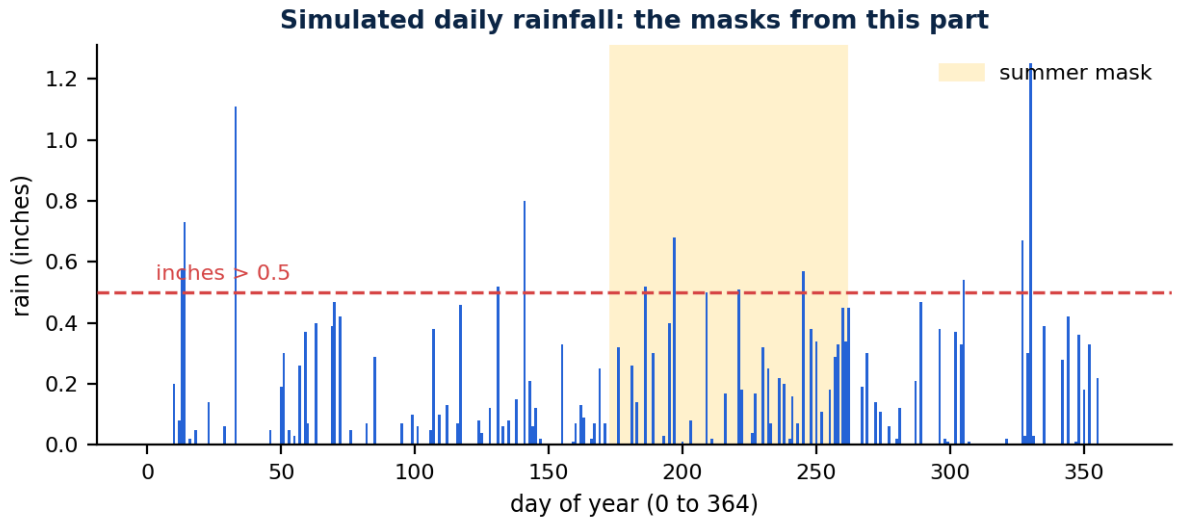
Long conditions are easier to read when you give the mask a name. Here is the book's rainfall idea. The book loads real Seattle data from a CSV file. We do not have that file, so we **simulate 365 days of rainfall** with a fixed seed. The numbers differ from the book, but every technique is identical.

PYTHON • type this in Colab

```
rng = np.random.default_rng(2014)
wet = rng.random(365) < 0.41
inches = np.where(wet, np.round(rng.exponential(0.25, 365), 2), 0.0)
print(inches.shape)
print(inches[:10])
```

▶ OUTPUT

```
(365,)
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```



Simulated rainfall for one year. The shaded band is the summer mask. The dashed line marks 0.5 inches.

Now build two masks and combine them. Every line reads almost like English:

PYTHON • type this in Colab

```
rng = np.random.default_rng(2014)
wet = rng.random(365) < 0.41
inches = np.where(wet, np.round(rng.exponential(0.25, 365), 2), 0.0)
rainy = (inches > 0)
days = np.arange(365)
summer = (days > 172) & (days < 262)

print("Number of rainy days: ", np.sum(rainy))
print("Days with more than 0.5 in: ", np.sum(inches > 0.5))
print("Median on rainy days: ", np.median(inches[rainy]))
print("Max on summer days: ", np.max(inches[summer]))
print("Median on non-summer rainy: ", np.median(inches[rainy & ~summer]))
```

▶ OUTPUT

```
Number of rainy days: 119
Days with more than 0.5 in: 12
Median on rainy days: 0.18
Max on summer days: 0.68
Median on non-summer rainy: 0.13
```

Notice the last line: `rainy & ~summer` means **rainy and not summer**. Masks combine exactly like any other Boolean arrays.

10.5 Selecting whole rows with a 1D mask

A 1D mask with one value **per row** selects entire rows of a 2D array. This is the standard way to pick the samples of one class **GOOD TO KNOW**:

PYTHON • type this in Colab

```
X = np.array([[1.0, 2.0],
              [3.0, 4.0],
              [5.0, 6.0],
              [7.0, 8.0]])
y = np.array([0, 1, 1, 0])
print(X[y == 1])
print(X[y == 0].mean(axis=0))
```

▶ OUTPUT

```
[[3. 4.]
 [5. 6.]]
[4. 5.]
```

`y == 1` has shape `(4,)`, one True or False per row. `X[y == 1]` keeps rows 1 and 2 with all their columns. The last line is the mean of each feature for class 0 only.

WHY IT MATTERS

`X[y == 1]` splits a dataset by class. `X[preds != y]` finds the samples a model got wrong, so you can study the mistakes. Pandas `df[df["age"] > 18]` is the same idea.

10.6 Changing values through a mask

A mask on the **left** of an assignment changes only the selected values:

PYTHON • type this in Colab

```
x = np.array([3, -7, 12, -5, 20])
x[x < 0] = 0
print(x)
```

▶ OUTPUT

```
[ 3 0 12 0 20]
```

This is how you write ReLU by hand, clip outliers, or replace invalid values. Remember that this **modifies** `x` **itself**. Take a copy first if you still need the original:

PYTHON • type this in Colab

```
x = np.array([3, -7, 12, -5, 20])
y = x.copy()
y[y < 0] = 0
print(x)
print(y)
```

▶ OUTPUT

```
[ 3 -7 12 -5 20]
[ 3 0 12 0 20]
```

10.7 x[mask] makes a copy

Reading with a mask gives you a **new array**, not a view of the original. Changing the result does not touch `x`:

PYTHON • type this in Colab

```
x = np.array([3, 7, 12, 5, 20])
y = x[x > 4]
y[0] = 99
print(y)
print(x)
```

▶ OUTPUT

```
[99 12 5 20]
[ 3 7 12 5 20]
```

COMMON TRAP

`y = x[mask]` **followed by** `y[0] = 99` **does not change** `x`**.** To change the original, put the mask on the left: `x[mask] = value`. Selecting makes a copy. Assigning through the mask edits `x`.

10.8 A mask of the wrong length

PYTHON • type this in Colab

```
x = np.array([3, 7, 12, 5, 20])
mask = np.array([True, False, True])
print(x[mask])
```

▶ OUTPUT

```
IndexError: boolean index did not match indexed array along axis 0; size of axis is 5 but size of corresponding boolean axis is 3
```

COMMON TRAP

The mask must match the array. A mask with 3 entries cannot select from an array with 5. Read the error message: it names both sizes. This usually happens when the mask was built from a different array.

✎ PRACTICE PROBLEMS | Part 10: Boolean Masks: Selecting Values

Q1 **PREDICT THE OUTPUT**

Predict both lines.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([10, -3, 25, 8, -12, 40, 0, 17])
print(x[x > 10])
print(x[(x >= 0) & (x < 20)])
```

Q2 **BUILD IT**

For `x = [10, -3, 25, 8, -12, 40, 0, 17]` (a) replace every negative value with 0 and print the array; (b) compute the mean of the **strictly positive** values of the **original** `x`.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([10, -3, 25, 8, -12, 40, 0, 17])
```

Q3 **PREDICT THE OUTPUT**

Predict the output and the shape of the result.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[5, 0, 3, 3],
              [7, 9, 3, 5],
              [2, 4, 7, 6]])
print(x[x < 5])
print(x[x < 5].shape)
```

Q4 **BUILD IT**

Given the data below, select (a) the rows of class 1, and (b) the mean of each feature over the class-0 rows.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1., 2.], [3., 4.], [5., 6.], [7., 8.], [9., 10.]])
y = np.array([0, 1, 1, 0, 1])
```

Q5 **FIND THE BUGS**

The student wants to set every value above 4 in `x` to 99, but `x` does not change. Explain and fix.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([3, 7, 12, 5, 20])
y = x[x > 4]
y[:] = 99
print(x)
```

Q6 **THINK IT THROUGH**

What happens here and why? What is the most likely cause of this error in real code?

PYTHON • type this in Colab

```
import numpy as np
x = np.array([3, 7, 12, 5, 20])
mask = np.array([True, False, True])
print(x[mask])
```

Q7 **BUILD IT**

Using the simulated rainfall below, compute (a) the number of rainy days, (b) the median rain on rainy days, (c) the maximum rain on summer days, (d) the median on rainy days that are **not** in summer.

PYTHON • type this in Colab

```
import numpy as np
rng = np.random.default_rng(2014)
wet = rng.random(365) < 0.41
inches = np.where(wet, np.round(rng.exponential(0.25, 365), 2), 0.0)
days = np.arange(365)
summer = (days > 172) & (days < 262)
```

Q8 **THEORY**

What is the difference between `np.sum(mask)` and `x[mask]`? Give an example of each, and say what shape `x[mask]` has when `x` is 2D. **[4 marks]**

PART 11

11 and / or versus & / |

Block 6, last section: whole object versus element by element

IN THIS PART YOU WILL LEARN

- ✓ The difference between the keywords `and`, `or` and the operators `&`, `|`
- ✓ Why `and` and `or` fail on NumPy arrays (the ValueError)
- ✓ What "ambiguous truth value" means and how `any()` / `all()` solve it
- ✓ The one rule to remember: arrays take `&`, `|`, `~`

11.1 Two kinds of "and"

Python has two ways to say "and" and two ways to say "or". They look similar but do different jobs:

Spelling	What it looks at	Gives you
`and`, `or`	the **whole object**: is this one thing True or False?	one single answer
`&`, `|`	the **parts inside** the object, one by one	one answer per element

11.2 On plain numbers

Python treats every non-zero number as True and 0 as False. `and` / `or` judge the **whole number**:

PYTHON • type this in Colab

```
print(bool(42), bool(0))
print(bool(42 and 0))
print(bool(42 or 0))
```

▶ OUTPUT

```
True False
False
True
```

`&` and `|` on integers work on the **bits** of each number instead (you do not need bit maths, only the idea that they compare positions):

PYTHON • type this in Colab

```
print(bin(42))
print(bin(59))
print(bin(42 & 59))
print(bin(42 | 59))
```

▶ OUTPUT

```
0b101010
0b111011
0b101010
0b111011
```

11.3 On NumPy Boolean arrays

A Boolean array is like a row of bits (True = 1, False = 0), so `&` and `|` work position by position, exactly as you used them in Part 9:

PYTHON • type this in Colab

```
A = np.array([1, 0, 1, 0, 1, 0], dtype=bool)
B = np.array([1, 1, 1, 0, 1, 1], dtype=bool)
print(A | B)
print(A & B)
```

▶ OUTPUT

```
[ True True True False True True]
[ True False True False True False]
```

But `or` tries to turn the **whole array** into a single True or False, and that is not defined:

PYTHON • type this in Colab

```
A = np.array([1, 0, 1, 0, 1, 0], dtype=bool)
B = np.array([1, 1, 1, 0, 1, 1], dtype=bool)
print(A or B)
```

▶ OUTPUT

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or
a.all()
```

The same thing happens when you use `and` between two comparisons:

PYTHON • type this in Colab

```
x = np.arange(10)
print((x > 4) and (x < 8))
```

▶ OUTPUT

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or
a.all()
```

IMPORTANT

"The truth value of an array with more than one element is ambiguous." NumPy is asking: do you mean "is *any* element True?" or "are *all* elements True?". There is no single sensible answer for a whole array, so it refuses. If you see this error, you used `and`, `or`, `not` or `if array:` on an array.

11.4 The fix, and the legitimate single answer

PYTHON • type this in Colab

```
x = np.arange(10)
print((x > 4) & (x < 8)) # element by element: correct mask
print(np.any((x > 4) & (x < 8))) # a single yes/no: is any element in range?
print(np.all(x < 10)) # a single yes/no: are all elements below 10?
```

▶ OUTPUT

```
[False False False False False True True True False False]
True
True
```

MEMORY TRICK

`and`/`or` **give ONE answer for the whole thing.** `&`/`|` **give one answer PER ELEMENT.** If you want a mask, use `&`/`|`. If you want one yes/no about the whole array, use `np.any` / `np.all`.

INTERVIEW FLAG

"What is the difference between `and` and `&` in Python/NumPy?"
Model answer: `and` evaluates the truth of a whole object and returns one result. `&` is the bitwise operator, which NumPy overloads to work **element by element** on Boolean arrays. On arrays `and` raises a ValueError (ambiguous truth value), so you must use `&` with brackets.

COMMON TRAP

`if x > 3:` **on an array raises the same ValueError.** Write `if np.any(x > 3):` or `if np.all(x > 3):` to say which meaning you want.

✎ PRACTICE PROBLEMS | Part 11: and / or versus & / |

Q1 THEORY

Explain the difference between `and` and `&` in two or three lines, and say which one works on a NumPy Boolean array. **[2 marks]**

Q2 PREDICT THE OUTPUT

Predict all five printed values.

PYTHON • type this in Colab

```
print(bool(42 and 0))
print(bool(42 or 0))
print(bool(0 or 7))
print(5 & 3)
print(5 | 3)
```

Q3 PREDICT THE OUTPUT

Predict both lines.

PYTHON • type this in Colab

```
import numpy as np
A = np.array([1, 0, 1, 0], dtype=bool)
B = np.array([1, 1, 0, 0], dtype=bool)
print(A | B)
print(A & B)
```

Q4 FIND THE BUGS

This line is meant to give a mask of the values strictly between 3 and 10. Say why it fails, then write the correct version and its output.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([2, 6, 9, 14, 3, 11])
print((x > 3) and (x < 10))
```

Q5 THINK IT THROUGH

In which situation is `np.any(x > 10)` the right tool, instead of `&` or `|`? Rewrite `if x > 10:` correctly for an array.

Q6 **BUILD IT**

A model outputs probabilities. Write one line that is True if **any** value is invalid (below 0 or above 1), and test it on the array below.

PYTHON • type this in Colab

```
import numpy as np
probs = np.array([0.2, 0.5, 1.3, 0.9])
```

PART 12

12 Revision Zone

Cheat sheet, interview quick-fire, error radar and a final mixed paper

IN THIS PART YOU WILL LEARN

- ✓ One-page function cheat sheet with priorities
- ✓ Topic checklist for Blocks 5 and 6, and what was left out on purpose
- ✓ 20 interview questions with model answers
- ✓ The error radar: symptoms, causes and fixes
- ✓ A final mixed paper covering broadcasting and masks together

12.1 Function cheat sheet

Group	Function / operator	What it does	Example	Priority
Broadcasting	the 3 rules	pad left with 1s; stretch the 1s; otherwise ValueError	`(2,3) + (3,)`	MUST KNOW
	`np.newaxis` / `None`	insert a size-1 axis	`a[:, np.newaxis]`	MUST KNOW
	`.reshape(-1, 1)`	turn a 1D array into a column	`w.reshape(-1, 1)`	MUST KNOW
	`keepdims=True`	keep the collapsed axis as size 1	`X.sum(axis=1, keepdims=True)`	MUST KNOW
	`np.linspace`	n evenly spaced numbers, both ends included	`np.linspace(0, 5, 50)`	GOOD TO KNOW
	`np.broadcast_to`	see the stretched array	`np.broadcast_to(a, (3, 3))`	LOOK UP
Comparison	`< > <= >= == !=`	Boolean array, same shape as input	`x < 3`	MUST KNOW
	`np.isclose`	compare floats safely	`np.isclose(a, b)`	MUST KNOW
	`np.less`, `np.equal` ...	the ufuncs behind the operators	`np.less(x, 3)`	LOOK UP
Count / check	`np.sum(mask)`	count the True values	`np.sum(x < 6)`	MUST KNOW
	`np.mean(mask)`	fraction of True values	`np.mean(preds == y)`	MUST KNOW
	`np.any` / `np.all`	at least one / every single one	`np.any(x > 8)`	MUST KNOW
	`np.count_nonzero`	count True (same as sum)	`np.count_nonzero(x < 6)`	GOOD TO KNOW
	`np.isnan`	Boolean mask of missing values	`np.any(np.isnan(X))`	MUST KNOW

Group	Function / operator	What it does	Example	Priority
Combine	`& | ~`	and, or, not (brackets round each comparison)	`(x > 4) & (x < 8)`	MUST KNOW
Select	`x[mask]`	the selected values (1D copy)	`x[x > 4]`	MUST KNOW
	`x[mask] = v`	change the selected values in place	`x[x < 0] = 0`	MUST KNOW
	`X[y == 1]`	select whole rows with a 1D mask	`X[y == 1]`	GOOD TO KNOW
	`.copy()`	independent copy of an array	`y = x.copy()`	MUST KNOW

12.2 Topic checklist

Topic from the blocks	Covered in	Done
Block 5: the idea of broadcasting (scalar, 1D + 2D)	Part 1	✓
Block 5: both arrays stretching, `np.newaxis`	Part 2	✓
Block 5: the three rules	Part 3	✓
Block 5: the book's three examples and reading the error	Part 4	✓
Block 5: fixing a failed broadcast	Part 5	✓
Block 5: centring data and the 2D function grid	Part 6	✓
Block 6: comparison operators as ufuncs	Part 7	✓
Block 6: counting and checking Boolean arrays	Part 8	✓
Block 6: Boolean operators `&`, `|`, `~`	Part 9	✓
Block 6: Boolean arrays as masks	Part 10	✓
Block 6: `and`/`or` versus `&`/`|`	Part 11	✓

Left out on purpose

These parts of the two blocks were skipped to keep the notes focused. Nothing here blocks you from the next sections.

Skipped	Why	When it returns
`np.logaddexp` example	only proves that every binary ufunc broadcasts	rarely typed by hand; search when needed
Bit-level detail of `&` and `|` on integers	you only need the idea: they work on parts, not the whole	low-level programming, if ever
Loading the Seattle CSV with pandas	we simulated the rainfall; the techniques are identical	Pandas phase
Histogram plotting code	plotting has its own phase	Matplotlib phase
`np.where(cond, a, b)`, `np.argwhere`	natural next step after masks	Fancy Indexing (next block) and Pandas
`np.meshgrid`	same job as the row + column trick, with explicit arrays	contour plots, loss surfaces

12.3 Interview quick-fire: 20 questions with model answers

#	Question	Model answer
1	What is broadcasting?	The rules NumPy uses to do element-wise operations on arrays of different shapes by virtually stretching the smaller one to match the larger.
2	State the three rules.	1) Pad the shape of the array with fewer dimensions with 1s on the left. 2) Stretch any dimension of size 1 to match the other array. 3) If sizes still differ and neither is 1, raise an error.
3	Does broadcasting copy the data?	No. The stretching is a mental model. NumPy reuses the same values, so it is fast and uses almost no extra memory.
4	Why does `(3, 2) + (3,)` fail?	`(3,)` pads on the left to `(1, 3)`. The last dimension is 2 against 3, neither is 1, so rule 3 applies.
5	What is `np.newaxis`?	An alias for `None` that inserts a new axis of size 1. `a[:, np.newaxis]` turns a 1D array into a column of shape `(n, 1)`.
6	How do you make a 1D array a column?	`a[:, np.newaxis]` or `a.reshape(-1, 1)`. Both give shape `(n, 1)`.
7	What does `keepdims=True` do?	It keeps the collapsed axis with size 1, so the aggregate can be broadcast back against the original array.
8	When can a broadcasting bug stay silent?	When the shapes happen to be compatible in the wrong direction, typically square arrays such as `(3, 3)` with `(3,)`: no error, wrong axis.
9	Result shape of `(4, 1) + (1, 6)`?	`(4, 6)`: the column stretches across and the row stretches down.
10	What does a comparison on an array return?	A Boolean array of the same shape as the input, one True or False per element.
11	How do you compute accuracy in one line?	`np.mean(preds == y)`: True counts as 1, so the mean is the fraction correct.
12	Why does `np.sum(mask)` count True values?	NumPy treats True as 1 and False as 0, so summing counts the Trues.
13	Difference between `np.any` and `np.all`?	`any` is True if at least one element is True. `all` is True only if every element is True. Both accept `axis`.

#	Question	Model answer
14	Why `np.sum` and not the built-in `sum`?	The built-in sum is for lists. On a 2D array it adds the rows and returns the wrong kind of answer without any error.
15	Why do you need brackets with `&`?	`&` binds tighter than `>` and `<`, so `x > 4 & x < 8` is parsed wrongly and fails. Write `(x > 4) & (x < 8)`.
16	Difference between `and` and `&` on arrays?	`and` needs one truth value for the whole array and raises a ValueError. `&` works element by element, which is what a mask needs.
17	What does `~` do and what is the trap?	Element-wise NOT on a Boolean array. On an integer array it flips bits (`~1` is `-2`), so apply it only to real Boolean masks.
18	Shape of `x[mask]` when `x` is 2D?	Always 1D: the kept elements cannot form a rectangle, so they come out flat in row-by-row order.
19	Does `x[mask]` copy? How do you edit `x`?	Reading with a mask makes a copy. To change the original, put the mask on the left: `x[mask] = value`.
20	Why not use `==` on floats?	Floats are stored approximately (`0.1 + 0.2` is not exactly `0.3`). Use `np.isclose`.

12.4 The error radar: check these before you trust your output

Symptom	Likely cause	What to do
`operands could not be broadcast together with shapes (a) (b)`	rule 3: a dimension clashes after left padding	Print both `.shape`s, pad left, find the clashing dimension, add `np.newaxis` or `keepdims=True`.
Code runs but numbers look wrong (often square arrays)	broadcast along the wrong axis	Check one element by hand. Use `keepdims=True` or `[:, np.newaxis]`.
`truth value of an array ... is ambiguous`	`and`, `or`, `not` or `if array:` used on an array	Use `&`, `|`, `~` with brackets, or `np.any` / `np.all`.
Error on `x > 4 & x < 8`	missing brackets	Wrap every comparison: `(x > 4) & (x < 8)`.
`~` gives `-2` and `-1`	`~` on an integer array flips bits	Use a real Boolean mask or `.astype(bool)`.
`sum(x < 6)` returns an array	built-in `sum` used on a 2D array	Use `np.sum`.
`boolean index did not match indexed array`	mask length differs from the array	Build the mask from the same array you index.
Original array unchanged after `y = x[mask]; y[0] = 99`	reading with a mask makes a copy	Edit through the mask on the left: `x[mask] = 99`.
`0.1 + 0.2 == 0.3` is False	float precision	Use `np.isclose`.
Original data lost after `x[x < 0] = 0`	assignment through a mask edits in place	Take `x.copy()` first if you need the original.

12.5 Final mixed paper

Everything from both blocks together. Try the whole paper before you open the answer key.

✎ PRACTICE PROBLEMS | Final Mixed Paper (Blocks 5 and 6)

Q1 **PREDICT THE OUTPUT**

Predict all three printed results.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([[1, 2, 3],
              [4, 5, 6]])
print(x - x.mean(axis=0))
print(x > x.mean(axis=1, keepdims=True))
print(np.sum(x % 2 == 0))
```

Q2 **FIND THE BUGS**

The student wants every row to sum to 1, but the row sums printed are not all 1, and no error appears. Explain and fix.

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1., 1., 2.],
              [2., 2., 4.],
              [3., 3., 3.]])
P = X / X.sum(axis=1)
print(P.sum(axis=1))
```

Q3 **BUILD IT**

Given the true labels and predictions below, compute (a) the accuracy, and (b) the positions of the mistakes (use a mask on `np.arange(6)`).

PYTHON • type this in Colab

```
import numpy as np
y_true = np.array([1, 0, 1, 1, 0, 1])
y_pred = np.array([1, 0, 0, 1, 1, 1])
```

Q4 **BUILD IT**

Flag the outliers in `x`: values more than 2 standard deviations away from the mean. Then replace each outlier by the mean of the non-outliers and print the cleaned array.

PYTHON • type this in Colab

```
import numpy as np
x = np.array([10., 12., 11., 13., 12., 95., 11., 10.])
```

Q5 **CALCULATE**

For each pair of shapes, give the result shape or say "error": (a) `(8, 1, 6, 1)` and `(7, 1, 5)`; (b) `(3,)` and `(4,)`; (c) `(5, 1)` and `(1, 5)`; (d) `(2, 3)` and `(2,)`.

Q6 **READ & DECODE**

Explain in plain words what this line returns and give the shape of the mask. Assume `X` has shape (n, 2).

PYTHON • type this in Colab

```
import numpy as np
X = np.array([[1, 3], [-2, 8], [4, 5], [6, 1]])
print(X[(X[:, 0] > 0) & (X[:, 1] < 5)])
```

Q7 **BUILD IT**

Standardise a random data set `X` of shape (200, 3), then count the values with absolute z-score above 2 in **each feature**, and tell whether any value exceeds 3.

PYTHON • type this in Colab

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.normal(size=(200, 3))
```

Q8 **BUILD IT**

Build the table of absolute differences between every pair of the points `a = [1, 4, 9]` without any loop.

PYTHON • type this in Colab

```
import numpy as np
a = np.array([1, 4, 9])
```

Q9 **THEORY**

Answer with examples: (i) the three broadcasting rules; (ii) the difference between `and` and `&` on arrays; (iii) why `y = x[mask]` does not change `x`. **[5 marks]**

ANSWER KEY

HOW TO USE **Try every question yourself first.** Then compare. If your answer is different, do not just copy mine: find the line where the two paths split. That is where the learning happens. Every output below was produced by running the code (NumPy 2.4.4).

Answers: Part 1: Broadcasting: The Big Idea

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[[10 20 30]
 [40 50 60]]
```

The scalar `10` was stretched into a 2×3 array of tens and multiplied element by element.

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
[101 102 103]
[[0 1]
 [2 3]]
```

In both lines a scalar is stretched to the shape of the array (`100` to 3 values, `1` to a 2×2 grid).

Q3 THINK IT THROUGH

None. The stretching is only a **mental model**: NumPy reuses the single value without allocating extra memory.

We still say "stretching" because it correctly predicts the **result**: the answer is exactly what you would get if the 5 really were repeated one million times.

Q4 PREDICT THE OUTPUT

▶ OUTPUT

```
[[11. 21. 31.]
 [11. 21. 31.]]
```

The 1D array `a` (shape `(3,)`, treated as `(1, 3)`) was stretched **down** to 2 rows.

Q5 BUILD IT

SOLUTION CODE

```
import numpy as np
Z = np.zeros((4, 3))
b = np.array([1, 2, 3])
print(Z + b)
```

▶ OUTPUT

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```

`b` has shape `(3,)`, which matches the last dimension of `Z`, so it is stretched down to 4 rows. This is exactly the bias pattern.

Q6 **FIND THE BUGS**

SOLUTION CODE

```
import numpy as np
Z = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([10, 20, 30])
print(Z + b)
```

▶ OUTPUT

```
[[11 22 33]
 [14 25 36]]
```

Two nested Python loops are exactly what Block 3 warned about. `Z + b` does the same job in one compiled operation (and keeps the integer dtype, while `np.empty` made floats).

Q7 **THEORY**

Broadcasting is how NumPy performs element-wise operations on arrays of different shapes: the smaller array is virtually stretched to match the larger one, without copying data.

ML use: adding a bias vector to every row of a layer output (`X @ W + b`); also subtracting a mean from every row when normalising data.

Answers: Part 2: Both Arrays Stretch

Q1 **PREDICT THE OUTPUT**

▶ OUTPUT

```
(3,) (2, 1)
[[11 12 13]
 [21 22 23]]
```

`a` is `(3,)`, which pads to `(1, 3)`; `b` is `(2, 1)`. The row stretches down to 2 rows and the column stretches across to 3 columns, so the result is `(2, 3)`.

Q2 **CALCULATE**

SOLUTION CODE

```
import numpy as np
a = np.arange(4)
print(a[:, np.newaxis].shape)
print(a[np.newaxis, :].shape)
```

▶ OUTPUT

```
(4, 1)
(1, 4)
```

`(4, 1)` is a column (new axis after the colon) and `(1, 4)` is a row (new axis before it).

Q3 **BUILD IT**

SOLUTION CODE

```
import numpy as np
n = np.arange(1, 6)
print(n[:, None] * n[None, :])
```

▶ OUTPUT

```
[[ 1 2 3 4 5]
 [ 2 4 6 8 10]
 [ 3 6 9 12 15]
 [ 4 8 12 16 20]
 [ 5 10 15 20 25]]
```

The column `(5, 1)` and the row `(1, 5)` stretch to `(5, 5)`, and cell `[i, j]` is `(i+1) * (j+1)`.

Q4 **THINK IT THROUGH**

`(3,)` is padded to `(1, 3)`, a row. `(3, 1)` is a column. A row and a column are different shapes, so **both** stretch: the row down, the column across.

Every row value meets every column value, which gives 9 sums, not 3.

Q5 **READ & DECODE**

SOLUTION CODE

```
import numpy as np
x = np.arange(5)
y = np.arange(7)
print((x[:, None] - y[None, :]).shape)
```

▶ OUTPUT

```
(5, 7)
```

It computes `x[i] - y[j]` for every pair, so the result is a `(5, 7)` table.

Q6 **FIND THE BUGS**

SOLUTION CODE

```
import numpy as np
a = np.arange(3)
b = np.arange(4)
print(a[:, None] + b)
```

▶ OUTPUT

```
[[0 1 2 3]
 [1 2 3 4]
 [2 3 4 5]]
```

Both are 1D, so they are lined up position by position, and `(3,)` against `(4,)` clashes (neither is 1). Turning `a` into a column `(3, 1)` makes it `(3,1) + (1,4)`, which stretches into `(3, 4)`.

Q7 **THEORY**

`np.newaxis` inserts a new axis of size 1. Example: `a` has shape `(3,)`; `a[:, np.newaxis]` has shape `(3, 1)` and `a[np.newaxis, :]` has shape `(1, 3)`.

Broadcasting stretches only axes of size 1 (and pads on the left), so `np.newaxis` lets you choose **which direction** an array should stretch.

Answers: Part 3: The 3 Rules of Broadcasting

Q1 CALCULATE

SOLUTION CODE

```
import numpy as np
for A, B in [((5, 3), (3,)), ((5, 3), (5,)), ((4, 1), (1, 6)), ((2, 3, 4), (3, 1))]:
    try:
        print(A, B, "->", np.broadcast_shapes(A, B))
    except ValueError:
        print(A, B, "-> error")
```

▶ OUTPUT

```
(5, 3) (3,) -> (5, 3)
(5, 3) (5,) -> error
(4, 1) (1, 6) -> (4, 6)
(2, 3, 4) (3, 1) -> (2, 3, 4)
```

(a) `(3,)` becomes `(1, 3)`; compatible; result `(5, 3)`.

(b) `(5,)` becomes `(1, 5)`; last dimension 3 vs 5, neither is 1; **not compatible**.

(c) already 2D; 4 vs 1 and 1 vs 6 both involve a 1; both stretch; result `(4, 6)`.

(d) `(3, 1)` becomes `(1, 3, 1)`; 4 vs 1, 3 vs 3, 2 vs 1 all pass; result `(2, 3, 4)`.

Q2 CALCULATE

SOLUTION CODE

```
import numpy as np
for A, B in [((4, 1), (4,)), ((1,), (5, 3)), ((8, 1, 6, 1), (7, 1, 5)), ((3, 4), (4, 3))]:
    try:
        print(A, B, "->", np.broadcast_shapes(A, B))
    except ValueError:
        print(A, B, "-> error")
```

▶ OUTPUT

```
(4, 1) (4,) -> (4, 4)
(1,) (5, 3) -> (5, 3)
(8, 1, 6, 1) (7, 1, 5) -> (8, 7, 6, 5)
(3, 4) (4, 3) -> error
```

(a) `(4,)` becomes `(1, 4)`. Then 4 vs 1 and 1 vs 4 both stretch, giving a **grid** `(4, 4)`. A column plus a 1D array is a classic surprise.

(b) `(1,)` becomes `(1, 1)` and stretches to `(5, 3)`; result `(5, 3)`.

(c) `(7, 1, 5)` becomes `(1, 7, 1, 5)`; compare from the right: 1 vs 5, 6 vs 1, 1 vs 7, 8 vs 1; result `(8, 7, 6, 5)`.

(d) 3 vs 4 in the first dimension, neither is 1: **error**.

Q3 THINK IT THROUGH

`(3,)` is padded on the **left** to `(1, 3)`, so its 3 is lined up against the **2** of `(3, 2)`, not against the 3. Last dimension: 2 vs 3, neither is 1, error.

Q4 **PREDICT THE OUTPUT**

▶ OUTPUT

```
(4, 4)
```

`(4, 1)` against `(1, 4)`: both stretch to `(4, 4)`. The shape you get is a grid, not a 4-element result.

Q5 **READ & DECODE**

Going from the right, each pair of sizes must be equal or one of them must be 1. The result takes the larger size in each position.

Q6 **THEORY**

Rule 1: pad the shape with fewer dimensions with 1s on the left. `(2,3)` and `(3,)` become `(2,3)` and `(1,3)`.

Rule 2: a dimension of size 1 stretches to match the other. `(2,3)` and `(1,3)` become `(2,3)` and `(2,3)`.

Rule 3: if sizes differ and neither is 1, it is a `ValueError`. `(3,2)` and `(1,3)` clash at 2 vs 3.

Answers: Part 4: The Rules on Real Code

Q1 **FIND THE BUGS**

▶ OUTPUT

```
ValueError: operands could not be broadcast together with shapes (4,3) (4,)
```

`(4,)` is padded to `(1, 4)`. Compared from the right: 3 vs 4, neither is 1, so it fails. The error message names both shapes, `(4,3)` and `(4,)`.

Q2 **PREDICT THE OUTPUT**

▶ OUTPUT

```
(3, 4)
[[0 1 2 3]
 [1 2 3 4]
 [2 3 4 5]]
```

`(3, 1)` against `(4,)` which pads to `(1, 4)`: both stretch to `(3, 4)`.

Q3 **READ & DECODE**

`(5,)` is padded to `(1, 5)`. The last dimension is 2 vs 5, neither is 1, so it clashes. The 5s match "by eye", but the padding went on the left.

Q4 **PREDICT THE OUTPUT**

▶ OUTPUT

```
[[4 5 4]
 [7 4 4]]
[[False True False]
 [ True False False]]
```

`a` is stretched down to 2 rows. `np.maximum` picks the larger of each pair; `M > a` gives a Boolean array of the same shape as `M`.

Q5 THINK IT THROUGH

It works with any binary ufunc: `M * a` (multiplying every row by a vector), `M > 5` (comparison with a scalar), `np.maximum(M, 0)` (the ReLU function), `x ** 2`, and so on.

Q6 THEORY

(1) print the `.shape` of both arrays; (2) pad the shorter shape with 1s on the left and compare from the right; (3) find the dimension where sizes differ and neither is 1, then use `np.newaxis` or `reshape` to line up the axis you meant.

Answers: Part 5: Fixing a Failed Broadcast

Q1 FIND THE BUGS

SOLUTION CODE

```
import numpy as np
M = np.array([[1, 2, 3],
              [4, 5, 6]])
a = np.array([10, 20])[:, np.newaxis]
print(M + a)
```

▶ OUTPUT

```
[[11 12 13]
 [24 25 26]]
```

(1) No: `(2,)` pads to `(1, 2)`, and the last dimension is 3 vs 2, which clashes. (2) and (3): make `a` a column, `(2, 1)`, so it stretches across 3 columns: row 0 gets +10, row 1 gets +20.

Q2 BUILD IT

SOLUTION CODE

```
import numpy as np
X = np.array([[1, 1], [2, 2], [3, 3]])
w = np.array([2, 3, 4])
print(X * w[:, np.newaxis])
```

▶ OUTPUT

```
[[ 2 2]
 [ 6 6]
 [12 12]]
```

`w` must become a column `(3, 1)` so it stretches across the 2 columns. Without `np.newaxis`, `(3,2)` against `(1,3)` would fail (2 vs 3).

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
[[1. 1. 1.]
 [2. 2. 2.]
 [3. 3. 3.]]
```

`X + b` lines `b` up with the **columns** (each column gets 1, 2, 3). `X + b.reshape(-1, 1)` lines `b` up with the **rows** (row 0 gets 1, row 1 gets 2, row 2 gets 3).

Q4 THINK IT THROUGH

For `(4,3)` with a length-4 vector, the wrong choice raises a ValueError, so you notice. For a square array both choices are valid shapes, so NumPy happily runs the wrong one and returns a plausible but wrong answer.

Q5 CALCULATE

SOLUTION CODE

```
import numpy as np
print(np.arange(6).reshape(-1, 1).shape)
print(np.arange(6).reshape(-1, 2).shape)
```

▶ OUTPUT

```
(6, 1)
(3, 2)
```

`-1` is worked out from the total of 6 elements: `(6, 1)` and `(3, 2)`.

Q6 THEORY

If the 1D array holds one value **per column**, use it as it is (it pads to `(1, n)` on the left). If it holds one value **per row**, convert it to a column with `a[:, np.newaxis]` or `a.reshape(-1, 1)`, shape `(n, 1)`.

Answers: Part 6: Broadcasting in Real Use

Q1 CALCULATE

SOLUTION CODE

```
import numpy as np
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
print(X.mean(axis=0), X.mean(axis=0).shape)
print(X - X.mean(axis=0))
print(X - X.mean(axis=1, keepdims=True))
```

▶ OUTPUT

```
[3. 4. 5.] (3,)
[[-2. -2. -2.]
 [ 0. 0. 0.]
 [ 2. 2. 2.]]
[[-1. 0. 1.]
 [-1. 0. 1.]
 [-1. 0. 1.]]
```

(1) `[3. 4. 5.]`, shape `(3,)`.

(2) `X - X.mean(axis=0)`: `(3,)` pads to `(1, 3)` and stretches down; each column ends with mean 0.

(3) `X.mean(axis=1)` has shape `(3,)`, which pads to a **row** `(1, 3)` and is copied down, so it subtracts `[2, 4, 6]` from every row (wrong axis). It runs only because the array is square. Fix: `keepdims=True` (or `[:, np.newaxis]`), giving a `(3, 1)` column that stretches across.

Q2 **BUILD IT**

SOLUTION CODE

```
import numpy as np
X = np.array([[1., 2., 3.],
              [3., 4., 5.],
              [5., 6., 7.]])
print(X / X.max(axis=0))
```

▶ OUTPUT

```
[[0.2 0.33333333 0.42857143]
 [0.6 0.66666667 0.71428571]
 [1. 1. 1. ]]
```

No. `X.max(axis=0)` has shape `(3,)`, which pads to `(1, 3)` and stretches down, so each column is divided by its own maximum. Left-padding already lines it up with the columns.

Q3 **BUILD IT**

SOLUTION CODE

```
import numpy as np
X = np.array([[1., 2., 1.],
              [2., 2., 4.],
              [5., 5., 0.]])
P = X / X.sum(axis=1, keepdims=True)
print(P)
print(P.sum(axis=1))
```

▶ OUTPUT

```
[[0.25 0.5 0.25]
 [0.25 0.25 0.5 ]
 [0.5 0.5 0. ]]
[1. 1. 1.]
```

The row totals are one value per row, so they must be a column: `keepdims=True` gives `(3, 1)`, which stretches across the columns. Each row of the result sums to 1.

Q4 **FIND THE BUGS**

SOLUTION CODE

```
import numpy as np
X = np.arange(12.).reshape(4, 3)
print(X - X.mean(axis=1, keepdims=True))
```

▶ OUTPUT

```
[[-1. 0. 1.]
 [-1. 0. 1.]
 [-1. 0. 1.]
 [-1. 0. 1.]]
```

`X.mean(axis=1)` has shape `(4,)`, which pads to `(1, 4)`, and the last dimension is 3 vs 4: clash. Here the mistake is loud (a non-square array); on a square array it would have been silent. The fix is `keepdims=True`.

Q5 CALCULATE

SOLUTION CODE

```
import numpy as np
x = np.linspace(0, 1, 50)
y = np.linspace(0, 1, 30)[:, np.newaxis]
print((x * y).shape)
```

▶ OUTPUT

```
(30, 50)
```

`(30, 1)` with `(50,)` which pads to `(1, 50)` gives `(30, 50)`. `z[i, j]` uses `y[i]` and `x[j]`: the row index goes with `y`, the column index with `x`.

Q6 READ & DECODE

It standardises each feature (column): subtract that column's mean and divide by that column's standard deviation, so each feature ends with mean 0 and standard deviation 1. `X` is `(samples, features)`; both statistics are `(features,)`, padded to `(1, features)` and stretched down over all samples.

Q7 THEORY

`keepdims=True` keeps the collapsed axis as size 1. For `X` of shape `(3, 3)`, `X.mean(axis=1)` has shape `(3,)` but `X.mean(axis=1, keepdims=True)` has shape `(3, 1)`.

The `(3, 1)` column can then be broadcast back against `X`, stretching across the columns, so each row has its own statistic applied.

Answers: Part 7: Comparisons Give Boolean Arrays

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[False False False True True True]
[ True True False True False True]
[ True True True False True True]
```

Each comparison is asked of every element. `x % 2 == 0` first computes the remainder array, then asks which remainders equal 0 (the even numbers).

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
[ True False True False]
0.5
```

It measures the accuracy: the fraction of predictions that match the true labels (2 correct out of 4 = 0.5).

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
[[False False False False]
 [ True False False False]
 [ True True True False]]
```

`t` has shape `(4,)`, padded to `(1, 4)` and stretched **down** to 3 rows. Column `j` of `x` is compared with `t[j]`.

Q4 READ & DECODE

▶ OUTPUT

```
[False True False True]
```

It thresholds the probabilities: every probability **strictly above** 0.5 becomes True (class 1), the rest False. Note the 0.5 itself gives False because the test is `>`, not `>=`.

Q5 FIND THE BUGS

▶ OUTPUT

```
False
```

SOLUTION CODE

```
import numpy as np
a = np.array([0.1, 0.2, 0.3])
print(np.isclose(a[0] + a[1], a[2]))
```

▶ OUTPUT

```
True
```

Floats are stored approximately, so `0.1 + 0.2` is `0.30000000000000004`, not exactly `0.3`. Compare floats with `np.isclose`.

Q6 CALCULATE

SOLUTION CODE

```
import numpy as np
m = np.arange(20).reshape(4, 5) >= 10
print(m.shape, m.dtype, m.sum())
```

▶ OUTPUT

```
(4, 5) bool 10
```

(a) `(4, 5)`, the same shape as the input. (b) `bool`. (c) The numbers 10 to 19 are True, so **10** values.

Q7 THEORY

`x < 3` works element by element on the whole array with no Python loop, exactly like `x + 3`: that is what makes it a universal function (ufunc) operation.

It calls `np.less(x, 3)`. The scalar 3 is broadcast to the shape of `x`. The result is an array of Booleans with the same shape as `x`.

Answers: Part 8: Counting and Checking

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
4
[2 0 2]
[False True False]
True
```

Values above 4: `5, 9` in row 0, none in row 1, `8, 6` in row 2, so the total is 4 and the row counts are `[2, 0, 2]`. Only the middle row is all 4s. At least one 4 exists, so `any` is True.

Q2 **PREDICT THE OUTPUT**

▶ **OUTPUT**

```
0.4444444444444444
[0.33333333 0.33333333 0.66666667]
```

The mean of Booleans is the fraction of True values (True = 1, False = 0). Overall 4 of 9 values exceed 4. Per column the fractions are 1/3, 1/3 and 2/3.

Q3 **BUILD IT**

SOLUTION CODE

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
print(np.sum(x > 3, axis=0))
print(np.any(np.all(x > 3, axis=1)))
```

▶ **OUTPUT**

```
[2 2 3]
True
```

(a) `np.sum(x > 3, axis=0)` counts down each column. (b) `np.all(x > 3, axis=1)` asks the question for each row, and `np.any(...)` of those answers says whether at least one row passes. Row 1 `[4, 4, 4]` passes, so the result is True.

Q4 **FIND THE BUGS**

▶ **OUTPUT**

```
[1 1 2]
```

SOLUTION CODE

```
import numpy as np
x = np.array([[1, 5, 9],
              [4, 4, 4],
              [8, 2, 6]])
print(np.sum(x > 4))
```

▶ **OUTPUT**

```
4
```

This is the built-in `sum`, which adds the rows together and gives one count per column. Use `np.sum`.

Q5 **BUILD IT**

SOLUTION CODE

```
import numpy as np
X = np.array([[1.0, np.nan, 3.0],
              [4.0, 5.0, np.nan],
              [np.nan, np.nan, 9.0]])
print(np.any(np.isnan(X)))
print(np.sum(np.isnan(X), axis=0))
```

▶ **OUTPUT**

```
True
[1 2 1]
```

`np.isnan(X)` gives a Boolean array. `np.any` of it answers (a). `np.sum(..., axis=0)` counts per column for (b): the columns have 1, 2 and 1 NaNs.

Q6 CALCULATE

SOLUTION CODE

```
import numpy as np
m = np.ones((6, 3), dtype=bool)
print(np.sum(m, axis=0).shape)
print(np.sum(m, axis=1).shape)
print(np.sum(m).shape)
```

▶ OUTPUT

```
(3,)
(6,)
()
```

`axis=0` removes the first dimension, leaving `(3,)`. `axis=1` leaves `(6,)`. With no axis everything collapses to one number, whose shape is `()`.

Q7 THINK IT THROUGH

`x < 6` is a Boolean array. NumPy treats True as 1 and False as 0, so adding the array adds one for every element that satisfies the condition, which is exactly the count.

Q8 THEORY

`np.any(mask)` is True if **at least one** element is True, e.g. `np.any(x < 0)` asks "is there a negative value?". `np.all(mask)` is True only if **every** element is True, e.g. `np.all(x < 10)` asks "are all values below 10?".

With `axis`, they answer the question for each row or column separately and return a Boolean array instead of one value, e.g. `np.all(x < 8, axis=1)` gives one answer per row.

Answers: Part 9: Combining Conditions

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[False False False False False True True True False False]
4
4
```

`(x > 4) & (x < 8)` is True for 5, 6, 7. `(x < 2) | (x > 7)` is True for 0, 1, 8, 9, which is 4 values. `~(x > 3)` is True for 0, 1, 2, 3, which is 4 values.

Q2 PREDICT THE OUTPUT

▶ OUTPUT

```
[False True False True False True False]
```

SOLUTION CODE

```
import numpy as np
x = np.array([3, 7, 12, 5, 20, 9, 1])
print(np.sum(~((x >= 5) & (x <= 10))))
```

▶ OUTPUT

```
4
```

The values inside the range are 7, 5 and 9 (three of them), so the count of values **outside** is `7 - 3 = 4`: the values 3, 12, 20 and 1.

Q3 **FIND THE BUGS**

▶ **OUTPUT**

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or
a.all()
```

SOLUTION CODE

```
import numpy as np
x = np.array([3, 7, 12, 5, 20, 9, 1])
print(np.sum((x > 4) & (x < 10)))
```

▶ **OUTPUT**

```
3
```

`&` binds tighter than `>` and `<`, so `4 & x` runs first and the rest is nonsense. Put each comparison in brackets. The answer is 3 (the values 7, 5 and 9).

Q4 **BUILD IT**

SOLUTION CODE

```
import numpy as np
x = np.array([10, -3, 25, 8, -12, 40, 0, 17])
print(np.sum((x < 0) | (x > 20)))
print(np.sum((x % 2 == 0) & (x > 0)))
```

▶ **OUTPUT**

```
4
3
```

(a) outside means below 0 or above 20: `-3, 25, -12, 40`, so 4. (b) even and positive: `10, 8, 40`, so 3 (0 is even but not positive).

Q5 **THINK IT THROUGH**

SOLUTION CODE

```
import numpy as np
x = np.arange(10)
print(np.array_equal((x > 4) & (x < 8), ~((x <= 4) | (x >= 8))))
```

▶ **OUTPUT**

```
True
```

It is De Morgan's law: "A and B" equals "not (not A or not B)". Here not A is `x <= 4` and not B is `x >= 8`.

Q6 **FIND THE BUGS**

▶ **OUTPUT**

```
[-2 -1 -2]
```

SOLUTION CODE

```
import numpy as np
flags = np.array([1, 0, 1])
print(~flags.astype(bool))
print(1 - flags)
```

▶ **OUTPUT**

```
[False True False]
[0 1 0]
```

On an integer array `~` flips the bits, so 1 becomes -2 and 0 becomes -1. Convert to bool first, or use `1 - flags` to stay with 0/1 numbers.

Q7 READ & DECODE

▶ OUTPUT

```
[ True False True True]
```

`X[:, 0]` is the first column and `X[:, 1]` the second. The mask is True for the **rows** where the first feature is positive **and** the second is at most 5. For a `(100, 2)` array the mask has shape `(100,)`: one answer per row.

Q8 THEORY

Because of operator precedence: `&` and `|` bind tighter than `>` and `<`, so without brackets Python tries `4 & x` before any comparison and fails.

Correct: `(x > 4) & (x < 8)`. Wrong: `x > 4 & x < 8`.

Answers: Part 10: Boolean Masks: Selecting Values

Q1 PREDICT THE OUTPUT

▶ OUTPUT

```
[25 40 17]
[10 8 0 17]
```

The mask keeps values in their original order. `x > 10` keeps 25, 40 and 17. `(x >= 0) & (x < 20)` keeps 10, 8, 0 and 17.

Q2 BUILD IT

SOLUTION CODE

```
import numpy as np
x = np.array([10, -3, 25, 8, -12, 40, 0, 17])
mean_pos = x[x > 0].mean()
x[x < 0] = 0
print(x)
print(mean_pos)
```

▶ OUTPUT

```
[10 0 25 8 0 40 0 17]
20.0
```

`x[x < 0] = 0` edits `x` in place. The mean of the positives must be taken **before** the edit, or from a copy, otherwise the zeros and the removed values change the picture. The positive values are 10, 25, 8, 40 and 17, whose mean is 20.0.

Q3 PREDICT THE OUTPUT

▶ OUTPUT

```
[0 3 3 3 2 4]
(6,)
```

The values below 5 are collected row by row: `0, 3, 3` from row 0, `3` from row 1, `2, 4` from row 2. That is 6 values in a **flat 1D** array.

Q4 **BUILD IT**

SOLUTION CODE

```
import numpy as np
X = np.array([[1., 2.], [3., 4.], [5., 6.], [7., 8.], [9., 10.]])
y = np.array([0, 1, 1, 0, 1])
print(X[y == 1])
print(X[y == 0].mean(axis=0))
```

▶ OUTPUT

```
[[ 3. 4.]
 [ 5. 6.]
 [ 9. 10.]]
[4. 5.]
```

`y == 1` is a mask with one value per row, so `X[y == 1]` keeps whole rows (rows 1, 2 and 4). `X[y == 0]` keeps rows 0 and 3, and `.mean(axis=0)` averages each column of those rows.

Q5 **FIND THE BUGS**

▶ OUTPUT

```
[ 3 7 12 5 20]
```

SOLUTION CODE

```
import numpy as np
x = np.array([3, 7, 12, 5, 20])
x[x > 4] = 99
print(x)
```

▶ OUTPUT

```
[ 3 99 99 99 99]
```

`x[x > 4]` on the right side makes a **copy**, so editing `y` never reaches `x`. Put the mask on the left of the assignment: `x[x > 4] = 99`.

Q6 **THINK IT THROUGH**

▶ OUTPUT

```
IndexError: boolean index did not match indexed array along axis 0; size of axis is 5 but size of
corresponding boolean axis is 3
```

The mask has 3 entries but `x` has 5, so NumPy cannot line them up and raises an IndexError that names both sizes. The usual cause is building the mask from a different array than the one you index.

Q7 **BUILD IT**

SOLUTION CODE

```
import numpy as np
rng = np.random.default_rng(2014)
wet = rng.random(365) < 0.41
inches = np.where(wet, np.round(rng.exponential(0.25, 365), 2), 0.0)
days = np.arange(365)
summer = (days > 172) & (days < 262)
rainy = inches > 0
print(np.sum(rainy))
print(np.median(inches[rainy]))
print(np.max(inches[summer]))
print(np.median(inches[rainy & ~summer]))
```

▶ OUTPUT

```
119
0.18
0.68
0.13
```

(a) `np.sum(rainy)`, (b) `np.median(inches[rainy])`, (c) `np.max(inches[summer])`, (d) `np.median(inches[rainy & ~summer])`. The real values are printed above. Masks and aggregates answer all four with no loop.

Q8 **THEORY**

`np.sum(mask)` **counts** how many elements satisfy the condition (a single number). `x[mask]` **returns the elements themselves** as a new array.

Example: for `x = [3, 7, 12]` and `mask = x > 4`, `np.sum(mask)` is 2 and `x[mask]` is `[7, 12]`. When `x` is 2D, `x[mask]` is always **1D** because scattered elements cannot keep a rectangular shape.

Answers: Part 11: and / or versus & / |

Q1 **THEORY**

`and` judges the truth of a **whole object** and returns one answer; `&` works **element by element** (bit by bit). On NumPy Boolean arrays you must use `&` (with brackets), because `and` raises a ValueError about an ambiguous truth value.

Q2 **PREDICT THE OUTPUT**

▶ OUTPUT

```
False
True
True
1
7
```

`42 and 0`: the second operand decides, so 0 gives False. `42 or 0` gives True. `0 or 7` gives True. For the bit patterns, `5 = 101` and `3 = 011`, so `5 & 3 = 001 = 1` and `5 | 3 = 111 = 7`.

Q3 **PREDICT THE OUTPUT**

▶ OUTPUT

```
[ True True True False]
[ True False False False]
```

`|` is True where at least one is True, `&` is True only where both are True, position by position.

Q4 **FIND THE BUGS**

▶ **OUTPUT**

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or
a.all()
```

SOLUTION CODE

```
import numpy as np
x = np.array([2, 6, 9, 14, 3, 11])
print((x > 3) & (x < 10))
```

▶ **OUTPUT**

```
[False True True False False False]
```

`and` tries to turn each whole array into one True/False, which raises the ambiguous-truth-value ValueError. `&` works element by element. Only 6 and 9 are strictly between 3 and 10.

Q5 **THINK IT THROUGH**

When you want **one yes/no answer about the whole array** ("is there at least one value above 10?") rather than a mask. `if x > 10:` fails on an array. Write `if np.any(x > 10):` for "at least one" or `if np.all(x > 10):` for "every one".

Q6 **BUILD IT**

SOLUTION CODE

```
import numpy as np
probs = np.array([0.2, 0.5, 1.3, 0.9])
print(np.any((probs < 0) | (probs > 1)))
```

▶ **OUTPUT**

```
True
```

`(probs < 0) | (probs > 1)` builds the mask of invalid values (brackets and `|`, not `or`), and `np.any` collapses it into one yes/no. Here 1.3 is invalid, so the answer is True.

Answers: Final Mixed Paper (Blocks 5 and 6)

Q1 **PREDICT THE OUTPUT**

▶ **OUTPUT**

```
[[-1.5 -1.5 -1.5]
 [ 1.5 1.5 1.5]]
[[False False True]
 [False False True]]
3
```

The column means `[2.5, 3.5, 4.5]` stretch down and are subtracted. The row means `[[2.], [5.]]` (a column, because of `keepdims`) stretch across, so each element is compared with its own row mean. Three numbers (2, 4, 6) are even.

Q2 FIND THE BUGS

▶ OUTPUT

```
[0.59722222 1.19444444 1.45833333]
```

SOLUTION CODE

```
import numpy as np
X = np.array([[1., 1., 2.],
              [2., 2., 4.],
              [3., 3., 3.]])
P = X / X.sum(axis=1, keepdims=True)
print(P.sum(axis=1))
```

▶ OUTPUT

```
[1. 1. 1.]
```

`X.sum(axis=1)` has shape `(3,)`, which pads to a row `(1, 3)`, so each **column** was divided by the wrong row total. It does not crash because the array is square. `keepdims=True` gives a `(3, 1)` column, which divides each row by its own total.

Q3 BUILD IT

SOLUTION CODE

```
import numpy as np
y_true = np.array([1, 0, 1, 1, 0, 1])
y_pred = np.array([1, 0, 0, 1, 1, 1])
print(np.mean(y_true == y_pred))
print(np.arange(6)[y_true != y_pred])
```

▶ OUTPUT

```
0.6666666666666666
[2 4]
```

(a) `np.mean(y_true == y_pred)`. (b) `y_true != y_pred` is True where the model was wrong; using it as a mask on `np.arange(6)` returns those positions (2 and 4).

Q4 BUILD IT

SOLUTION CODE

```
import numpy as np
x = np.array([10., 12., 11., 13., 12., 95., 11., 10.])
out = np.abs(x - x.mean()) > 2 * x.std()
print(out)
x[out] = x[~out].mean()
print(x)
```

▶ OUTPUT

```
[False False False False False True False False]
[10. 12. 11. 13. 12. 11.28571429
 11. 10. ]
```

`x - x.mean()` broadcasts a scalar, `np.abs` removes the sign, and the comparison builds the outlier mask. `x[~out].mean()` is the mean of the values that are not outliers, and `x[out] = ...` writes it into the flagged positions.

Q5 CALCULATE

SOLUTION CODE

```
import numpy as np
pairs = [((8, 1, 6, 1), (7, 1, 5)), ((3,), (4,)), ((5, 1), (1, 5)), ((2, 3), (2,))]
for a, b in pairs:
    try:
        print(a, b, "->", np.broadcast_shapes(a, b))
    except ValueError:
        print(a, b, "-> error")
```

▶ OUTPUT

```
(8, 1, 6, 1) (7, 1, 5) -> (8, 7, 6, 5)
(3,) (4,) -> error
(5, 1) (1, 5) -> (5, 5)
(2, 3) (2,) -> error
```

(a) pad `(7, 1, 5)` to `(1, 7, 1, 5)`; every dimension passes, result `(8, 7, 6, 5)`. (b) 3 vs 4, error. (c) column and row give `(5, 5)`. (d) `(2,)` pads to `(1, 2)`, and the last dimension is 3 vs 2: error.

Q6 READ & DECODE

▶ OUTPUT

```
[[1 3]
 [6 1]]
```

It returns the **rows** whose first feature is positive and whose second feature is below 5. The mask has shape `(n,)`, one True/False per row, and the result keeps whole rows, so it is 2D with 2 columns.

Q7 BUILD IT

SOLUTION CODE

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.normal(size=(200, 3))
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(np.sum(np.abs(Z) > 2, axis=0))
print(np.any(np.abs(Z) > 3))
```

▶ OUTPUT

```
[9 7 6]
True
```

The statistics have shape `(3,)`, which pads to `(1, 3)` and stretches down over 200 samples. `np.sum(..., axis=0)` counts per feature, and `np.any` gives one yes/no for the whole array.

Q8 **BUILD IT**

SOLUTION CODE

```
import numpy as np
a = np.array([1, 4, 9])
D = np.abs(a[:, np.newaxis] - a)
print(D)
```

▶ OUTPUT

```
[[0 3 8]
 [3 0 5]
 [8 5 0]]
```

`a[:, np.newaxis]` has shape `(3, 1)` and `a` has shape `(3,)`, which pads to `(1, 3)`. The column stretches across and the row stretches down, giving a `(3, 3)` table where `D[i, j]` is `|a[i] - a[j]|`.

Q9 **THEORY**

(i) Pad the smaller shape with 1s on the left; stretch dimensions of size 1; otherwise raise an error. Example: `(2, 3)` with `(3,)` becomes `(2, 3)` with `(1, 3)`, then both `(2, 3)`.

(ii) `and` needs one truth value for the whole array and raises a ValueError; `&` works element by element and needs brackets: `(x > 4) & (x < 8)`.

(iii) Selecting with a mask makes a **copy**. To modify `x`, put the mask on the left side: `x[mask] = value`.